

claude-windows / f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb\subagents\workflows\wf\_1ee5ffdf-362\agent-a04389dd389ee6ddb.jsonl`
- SHA-256: `df57910d9460c1c79bd95f58a0d7a5b309c0752c8ddc196c72b93a324228d3cf`
- Source modified: `2026-06-22T16:17:31+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf\_1ee5ffdf-362`
- session_id: `f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb`

Transcript

1. user (2026-06-22T16:12:14.849Z)

Review the UNCOMMITTED M11 change in the git worktree at C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\m11-flywheel (run 'git -C "C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\m11-flywheel" diff' + read the two new files backend/flywheel.py and tests/test_flywheel.py).
WHAT IT IS: a default-OFF, privacy-sensitive "data flywheel" skeleton that, when activated, would CAPTURE a user's uploaded video + rally output into a per-video workspace for later use as TRAINING data. It MUST be impossible for it to capture anything in production today.
The design: backend/flywheel.py has consent_attested() (a FAKED hard-deny returning False always), capture_job(), shadow_eval() (delegates to backend.eval.experiment.compare/compare_unlabeled), propose_promotion() (delegates to backend.eval.promotion.promote_if_served_safe), and run_for_job() (the gated orchestrator).
FlywheelConfig (enabled=False default) added to AppConfig. backend/api/job_worker.py got a gated _maybe_run_flywheel hook (mirrors the M8 _maybe_record_job_cost) called at job completion. No DB migration.
Read for grounding: backend/flywheel.py, backend/api/job_worker.py (the hook + the M8 cost hook it mirrors), backend/config/models.py (FlywheelConfig + AppConfig), backend/utils/workspace.py (VideoWorkspace.outputs/source are PROPERTIES; for_video; resolve_workspace_root), backend/eval/experiment.py (compare/compare_unlabeled signatures), backend/eval/promotion.py (promote_if_served_safe ? confirm the A/B argument order). Be adversarial; default to a finding if uncertain.

LENS = CORRECTNESS / DELEGATION / DEFAULT-OFF. Verify:

1. The wrappers delegate correctly: shadow_eval ? experiment.compare (labeled) / compare_unlabeled (unlabeled, single video); does the arg order + signature match the real functions? propose_promotion ? promote_if_served_safe ? CONFIRM the A/B order is correct (read promotion.py: which arg is control vs proposal, and does flywheel pass Gemini as A=control and owned as B=proposal? a wrong order would invert the promotion gate).
 2. VideoWorkspace.outputs/source are PROPERTIES (not methods) ? confirm flywheel.py accesses them WITHOUT parens (a prior bug). resolve_workspace_root + for_video used correctly?
 3. capture_job: the tempfile manifest write + storage.put + the finally-unlink ? any leak/exception path? Does it handle a missing source_path gracefully? Is the storage config dict conversion (.sc) correct vs what backend.storage.put expects?
 4. DEFAULT-OFF: AppConfig() with no flywheel block ? flywheel.enabled False ? byte-identical to today. Does adding the FlywheelConfig field break any existing config test (e.g. a test asserting AppConfig's exact field set)? Does any existing test of job_worker now exercise the new hook?
 5. Is NO DB migration correct (the skeleton persists no consent)? Confirm it doesn't reference any non-existent column.
- Return findings with file:line + a ship verdict.

2. assistant (2026-06-22T16:12:17.313Z)

I'll analyze this M11 flywheel change for correctness, delegation, and default-OFF safety. Let me start by examining the uncommitted changes and then read the relevant files.

3. user (2026-06-22T16:12:17.996Z)

```
diff --git a/backend/api/job_worker.py b/backend/api/job_worker.py
index 50642ed..aae876c 100644
--- a/backend/api/job_worker.py
+++ b/backend/api/job_worker.py
@@ -89,6 +89,7 @@ def _run_claimed_job(job: Dict[str, Any]) -> None:
    db_instance.set_job_video_id(job_id, result["video_id"])
    db_instance.mark_job_done(job_id, result)
    _maybe_record_job_cost(job, result) # M8 cost ledger (default-OFF)
+    _maybe_run_flywheel(job, result)    # M11 data-flywheel (default-OFF + consent-faked-
deny)
    except _main.JobWaiting as w:
        # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
        logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
@@ -129,6 +130,28 @@ def _maybe_record_job_cost(job: Dict[str, Any], result: Dict[str, Any]) ->
None:
    logger.warning("Cost recording for job %s failed (non-fatal): %s", job.get("id"), e)

+def _maybe_run_flywheel(job: Dict[str, Any], result: Dict[str, Any]) -> None:
+    """M11 data-flywheel (COMPUTE_DECOUPLED_SERVING M11, D12). DEFAULT-OFF + INERT: a no-op
unless
+    ``config.flywheel.enabled`` AND attested consent ? and consent is a **faked hard-deny**
today
+    (``flywheel.consent_attested`` returns False for every job until counsel + the consent
schema
+    land), so this captures **nothing** in production. When activated it captures the consented
job's
+    artifacts into the per-video workspace (? later labeling ? the golden training set).
+
+    Best-effort ? never fails a completed job because the flywheel hiccuped. ``job`` is the
claimed
+    row (carries ``owner_id``); ``result`` is the pipeline output."""
+    import backend.main as _main
+
+    flywheel_cfg = getattr(_main.global_config, "flywheel", None)
+    if not flywheel_cfg or not getattr(flywheel_cfg, "enabled", False):
+        return
+    try:
+        from backend import flywheel
+
+        flywheel.run_for_job(job, result, _main.global_config)
+    except Exception as e: # never wedge a completed job on the flywheel
+        logger.warning("Flywheel for job %s failed (non-fatal): %s", job.get("id"), e)
+
+def _drain_due_jobs() -> int:
+    """One worker tick: claim and run every currently-runnable job (queued, or waiting whose
`next_poll_at` is due), earliest-due first, until the table has nothing runnable.
diff --git a/backend/config/models.py b/backend/config/models.py
index d77f98e..9c8cf77 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -384,6 +384,26 @@ class BillingConfig(BaseModel):
    margin: float = 0.0                                # resale markup (?0); 0 = bill at cost

+class FlywheelConfig(BaseModel):
+    """Data-flywheel harness (COMPUTE_DECOUPLED_SERVING M11, D12) ? the
consent?capture?eval?goldenize
+    loop: on a CONSENTED adult job, durably **capture** the source + rally output into the per-
video
+    workspace ? **shadow-eval** owned-vs-Gemini ? **propose promotion** to the golden TRAINING
set, so
```

```

+   the owned model climbs toward the D11 ship floor.
+
+   **DEFAULT-OFF + INERT:** ``enabled=False`` ? nothing is captured/evaluated (byte-identical
to
+   today). ? **Consent is FAKED-DENY until counsel + the consent schema land:** even
``enabled=True``
+   captures **nothing**, because ``flywheel.consent_attested`` currently returns ``False`` for
every
+   job (the privacy guardrail ? D10: adults-only, explicit opt-in, derived-data + auto-delete-
raw).
+   This is the PLUMBING only; the real consent gate (counsel-reviewed ToS/DPA ? consent
columns ?
+   wiring) + the owned-model + golden-label data needed to *activate* shadow-eval/promotion
are
+   owner/counsel/GPU scope. Pairs with M9-consent.""
+
+   enabled: bool = False
+   workspace_prefix: str = "flywheel"      # per-video workspace prefix for captured artifacts
+   shadow_eval: bool = True                 # run the owned-vs-Gemini shadow comparison on capture
+   min_promote_n: int = 5                   # promote_if_served_safe `min_n` (the served-safe
gate)
+
+
+   class AppConfig(BaseModel):
+       """Root configuration object.

@@ -406,6 +426,7 @@ class AppConfig(BaseModel):
    storage: StorageConfig = Field(default_factory=StorageConfig)
    deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
    billing: BillingConfig = Field(default_factory=BillingConfig)
+   flywheel: FlywheelConfig = Field(default_factory=FlywheelConfig)

    # Allow extra keys during transition so existing config.json files do not break.
    model_config = {
diff --git a/docs/COMPUTE_DECOUPLED_SERVING/README.md b/docs/COMPUTE_DECOUPLED_SERVING/README.md
index ec9f84b..1576e75 100644
--- a/docs/COMPUTE_DECOUPLED_SERVING/README.md
+++ b/docs/COMPUTE_DECOUPLED_SERVING/README.md
@@ -123,7 +123,7 @@ Legend: ? Todo . ? In progress . ? Done . ? Gated. **One small PR per
| M8 | Per-job **cost ledger** (v6 migration: `owner_id, gpu_active_seconds, wall_seconds,
bytes_out, cost_usd, cost_breakdown_json`) + **pricebook** + aggregation + `/api/.../cost` | M7
| ? owner (billing) | ? | [#288](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/288) (ledger+pricebook+/cost ? default-OFF; real prices + usage-emission ?
owner/follow-up) |
| M9 | **Edge auth** (Cf-Access extraction) + per-tenant scoping on `user_id`; DPA/consent gate
wiring | M7 | ? owner (privacy/counsel) | ? | [#290](https://github.com/Khelsutra/badminton-
highlight-indexer/pull/290) (auth: Cf-Access JWT verify + owner_id tag, default-OFF ?; **+
exposure-safety boot posture** in `startup.py` ? edge-auth-on requires the `allowed_video_root`
jail + CF team/AUD or the server refuses to start, see changelog; consent cols + ToS/DPA ?
owner/counsel) |
| M10 | `byo_worker` / zero-infra-BYO ? **design-only, DEFERRED** | M8,M9 | ? explicit owner
approval | ? | ? |
-| **M11** | **Data-flywheel harness (D12):** on a consented adult job, **capture** the upload +
Gemini reel/rally output ? **reliably store** under the per-video workspace ? **shadow-eval**
owned-vs-Gemini (dual-eval-set / `experiment.compare`) ? **promote** good ones to the golden
TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the owned model
climbs to the D11 floor. Reuses `data_pipeline/` + `backend/eval/` + the consent gate (M9). |
M7,M9 | ? owner (consent/privacy) | ? | ? |
+| **M11** | **Data-flywheel harness (D12):** on a consented adult job, **capture** the upload +
Gemini reel/rally output ? **reliably store** under the per-video workspace ? **shadow-eval**
owned-vs-Gemini (dual-eval-set / `experiment.compare`) ? **promote** good ones to the golden
TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the owned model
climbs to the D11 floor. Reuses `data_pipeline/` + `backend/eval/` + the consent gate (M9). |
M7,M9 | ? owner (consent/privacy) | ? | ? | **skeleton landed** (see changelog):
`backend/flywheel.py` + `FlywheelConfig` (default-OFF) + gated `_maybe_run_flywheel` hook,

```

```

**consent faked-deny**; live activation ? owner/counsel (consent schema) + owned-model/golden
data |
| **M12** | **`gdrive-api` (OAuth) StorageBackend (D14 Northstar):** cloud reads the source
from + writes the stitched reel **back to the user's Google Drive next to the source** (per-user
OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local mount
backend; slots into the `StorageBackend` ABC. | M7,M9 | ? owner (OAuth/consent) | ? | ? |

**Testing strategy is a first-class deliverable** ?
[`TESTING_STRATEGY.md`](TESTING_STRATEGY.md). **Run logs / sample runs for posterity** ?
[`runlogs/`](runlogs/).
@@ -168,3 +168,4 @@ Legend: ? Todo . ? In progress . ? Done . ? Gated. **One small PR per
- 2026-06-21 ? **M6 code landed** ([#287](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/287)): `backend/compute/` ComputeBackend registry ? `CloudRunCompute` dispatches a
Cloud Run **Job** (injectable client) then **bucket-polls** a done-marker via
`execution_policy.poll_until` + the storage layer (D16); `get_compute_backend` returns a backend
ONLY for `compute_target=cloud_run` (else `None` ? the worker's in-process path, **default-
OFF**). The dispatched container runs **inline** (forced `local_gpu`+native, never re-dispatch);
the worker writes a `--done-uri` completion marker on success.
`deploy/cloudrun/{Dockerfile,README}` = the L4 image (CUDA+ffmpeg+fonts+`wasb` conda env;
weights staged per WASB-C3) + the M7 runbook. Adversarial review (`wf_87b22fa4`) folded a
**blocker** (cloud `fetch` needs a dst ? reads silently returned `{}` on gcs/s3) + a poll-wait
test gap + `PolicyTimeout`?clean rc 4. Suite green, ruff+mypy clean. **? M6 ? ? the real
build/push/L4 run + DEPLOY_REPORT is M7 (owner, D17 budget).**
- 2026-06-21 ? **M5 code landed** ([#286](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/286)): `backend/config/startup.py` ? per-profile fail/warn-fast startup checks
(cloud-serving + non-cloud storage = the one fatal coherence error; GPU/creds mismatches =
warnings, the runner healthcheck stays the authority), wired into the server
(`_enforce_startup_or_exit` ? exit 1) + the worker (`cmd_infer`/`cmd_train` ? rc 2), default-OFF
(probe gated behind an active profile ? the worker stays torch-free at `profile=""`). **L4
profile-parity test** (`tests/test_profile_parity.py`): same stub trajectory ? byte-identical
windows + segment ranges across truly-local / cpu-mock / no-profile + worker-vs-in-process.
Adversarial 4-lens review folded (`wf_34671ac1`). Suite green, ruff+mypy clean. **? M5 ? ? the
real-GPU truly-local runlog (L5) is owner-side** (template pre-staged in `runlogs/`).
- 2026-06-21 ? **M9 hardening: exposure-safety boot posture** (alpha-shareable "tunnel-to-a-
box" enabler): extended `backend/config/startup.py` with a **server-only exposure posture** (new
`include_exposure=True` from `_enforce_startup_or_exit`; the worker stays unaffected ? it reads
`--video-uri`, not untrusted tenant paths). Gated on `security.edge_auth_enabled` (**default-OFF
? strict no-op**, byte-identical to today): when the owner flips edge auth ON to expose the
engine behind the CF-Access gate, three checks fire at **boot** ? `EXPOSED_NO_VIDEO_ROOT`
(**fatal**: `allowed_video_root` jail unset ? a tenant could read an arbitrary local file),
`EXPOSED_EDGE_AUTH_INCOMPLETE` (**fatal**: CF team-domain/AUD missing ? lifts the current first-
request 401 to boot), `EXPOSED_AUTH_EXTRA_MISSING` (**warn**: `PyJWT[crypto]` not importable ?
would 500 per request). So a half-configured exposure fails fast instead of leaking/500-ing. +15
unit tests (default-OFF no-op, worker-ignores, both-fatals-compose-with-profile-checks, server-
exit, WARN-only-no-exit). Adversarial 3-lens review (`wf_35b735a9`) confirmed default-OFF byte-
identical + no bypass; folded a **whitespace-drift** (aligned `auth.resolve_tenant` to
`.strip()` team/AUD so "configured" means the same at boot + at request time) + named the config
fields in the edge-auth-incomplete message. Pairs with the appliance **T3** network posture (raw
`8000` refused ? owner-run). Suite green, ruff+mypy clean. **? backstops M9 ? the actual
exposure flip + CF team/AUD/ingress + key rotation stay owner-side. (Deferred to the PR-2
runbook: document that `upload_dir` must sit under `allowed_video_root`, and that you MUST set
`edge_auth_enabled=True` when exposing ? these checks only fire then.)**
+- 2026-06-22 ? **M11 skeleton landed** (data-flywheel harness, D12): `backend/flywheel.py`
(`capture_job` ? per-video `VideoWorkspace` via `backend.storage.put`; `shadow_eval` ?
`experiment.compare`/`compare_unlabeled`; `propose_promotion` ?
`promotion.promote_if_served_safe` with Gemini=control/owned=proposal; `run_for_job`
orchestrator) + `FlywheelConfig` (**default-OFF**) + a gated `_maybe_run_flywheel` job-
completion hook mirroring M8's cost hook. **TWO independent guards keep it INERT in
production:** `flywheel.enabled=False` (default) AND `consent_attested` is a **FAKED hard-deny**
(returns False for every job) ? so even enabled it captures **nothing** until the real consent
schema lands. This is the PLUMBING only; it reuses the real workspace/eval/promotion/storage
code so it works the day consent + the owned model + golden labels are wired. **NO DB
migration** (the skeleton persists no consent). +13 tests (default-OFF no-op, enabled-but-
consent-denied no-op, consented-capture writes the manifest, delegation of shadow-eval/promote,
the hook is best-effort). Suite green, ruff+mypy clean. **? M11 ? ? LIVE activation is

```

owner/counsel (the consent ToS/DPA + schema ? the consent columns) + owner/GPU (the owned model + golden corpus for shadow-eval/promotion). ? the future M9-consent migration is now `v8` (v7 was taken by the `locale` column).**

4. assistant (2026-06-22T16:12:20.146Z)

Now let me read the new files and the relevant existing code to perform a thorough analysis:

5. user (2026-06-22T16:12:21.532Z)

```
1      """Data-flywheel harness (COMPUTE_DECOUPLED_SERVING M11, D12) ? capture ? shadow-eval ?
promote.
2
3      The consent?capture?eval?goldenize loop: on a **consented adult** job, durably
**capture** the source
4      + rally output into the per-video workspace ? **shadow-eval** the owned model vs Gemini
? **propose
5      promotion** of good ones into the golden TRAINING set, so the owned model climbs to the
D11 ship floor.
6
7      ? **DEFAULT-OFF + INERT ? two independent guards keep this from doing ANYTHING in
production today.**
8      1. ``config.flywheel.enabled`` defaults ``False`` ? the job-completion hook is a
strict no-op.
9      2. :func:`consent_attested` is a **FAKED hard-deny** (returns ``False`` for every job)
until the real
10     consent schema lands (counsel-reviewed ToS/DPA ? consent columns ? wiring). So even
11     ``enabled=True`` captures **nothing**. This is the privacy guardrail ? D10 + the
CLAUDE.md rule:
12     adults-only, explicit opt-in, train-on-derived + auto-delete-raw; **minors
excluded**; per-user
13     training is a *separate* opt-in; paid tier never trains.
14
15     This module is the PLUMBING ? it REUSES the real ``workspace`` / ``eval.experiment`` /
16     ``eval.promotion`` / ``storage`` code so the loop works the day consent + the owned
model + golden
17     labels are wired. It ships **no live behavior change**. Pairs with M9-consent (the
consent columns).
18     """
19     from __future__ import annotations
20
21     import json
22     import logging
23     import os
24     import tempfile
25     from typing import Any, Dict, Optional, Sequence
26
27     from backend import storage as _storage
28     from backend.utils import workspace as _workspace
29
30     logger = logging.getLogger(__name__)
31
32
33     def _sc(storage_config: Any) -> Optional[Dict[str, Any]]:
34         """Normalize an ``AppConfig.storage`` (pydantic model / dict / None) to the dict
35         ``backend.storage`` expects."""
36         if storage_config is None:
37             return None
38         if hasattr(storage_config, "model_dump"):
39             return storage_config.model_dump()
40         return storage_config if isinstance(storage_config, dict) else None
41
42
43     def consent_attested(job: Dict[str, Any], config: Any = None) -> bool:
44         """The flywheel's consent gate ? currently a **FAKED hard-deny** (always ``False``).
```

```

45
46         The flywheel may capture a user's footage into the training pool ONLY with attested,
**adults-only**
47         consent (D10 / the CLAUDE.md guardrail: minors excluded; per-user training is a
separate explicit
48         opt-in; paid tier never trains). The REAL gate ? reading counsel-defined consent
columns + an
49         adults-only attestation + a retention class ? lands with **M9-consent**; until then
this denies
50         every job so the harness captures nothing. **Do not relax without counsel sign-
off.**""""
51         return False
52
53
54     def capture_job(
55         workspace: "_workspace.VideoWorkspace",
56         *,
57         rally_output: Dict[str, Any],
58         source_path: Optional[str] = None,
59         storage_config: Any = None,
60     ) -> Dict[str, str]:
61         """Durably persist a consented job's artifacts into its per-video ``workspace`` (the
raw material a
62         human later labels ? the golden training set). Always writes a small
``capture.json`` manifest
63         (``video_id`` + rally output + provenance); when a local ``source_path`` is given,
also stores the
64         source video. Returns the written URIs. Reuses :func:`backend.storage.put` + the
``VideoWorkspace``
65         layout, so it honours whatever ``storage.backend`` the deploy uses (local / gcs /
s3)."""
66         written: Dict[str, str] = {}
67         cfg = _sc(storage_config)
68         fd, manifest_local = tempfile.mkstemp(suffix=".json")
69         try:
70             with os.fdopen(fd, "w", encoding="utf-8") as fh:
71                 json.dump({"video_id": workspace.video_id, "rally_output": rally_output},
fh, indent=2)
72                 manifest_uri = _workspace.join_uri(workspace.outputs, "capture.json")
73                 written["manifest_uri"] = _storage.put(manifest_local, manifest_uri, config=cfg)
74                 if source_path and os.path.exists(source_path):
75                     source_uri = _workspace.join_uri(workspace.source,
os.path.basename(source_path))
76                     written["source_uri"] = _storage.put(source_path, source_uri, config=cfg)
77             finally:
78                 try:
79                     os.unlink(manifest_local)
80                 except OSError:
81                     pass
82         return written
83
84
85     def shadow_eval(
86         candidates: Sequence[Any],
87         owned_variant: Any,
88         gemini_variant: Any,
89         *,
90         labeled: bool = False,
91         iou: float = 0.5,
92     ) -> Dict[str, Any]:
93         """Owned-vs-Gemini shadow comparison (the D11 climb signal). Delegates to the
experiment harness:
94         a labeled golden corpus ? :func:`backend.eval.experiment.compare` (B?A deltas +
honest CIs); an
95         unlabeled single video ? :func:`backend.eval.experiment.compare_unlabeled`

```

```

(agreement). The owned
96     model + a golden corpus are owner/GPU-side, so this only *runs* once they exist; the
wrapper makes
97     the call site real today (and is exercised via tests with stub variants)."""
98     from backend.eval import experiment
99
100    if labeled:
101        return experiment.compare(candidates, owned_variant, gemini_variant, iou=iou)
102    if len(candidates) != 1:
103        raise ValueError("unlabeled shadow_eval compares exactly one video at a time")
104    return experiment.compare_unlabeled(candidates[0], owned_variant, gemini_variant,
agree_iou=iou)
105
106
107    def propose_promotion(
108        served_owned: Sequence[float],
109        served_gemini: Sequence[float],
110        *,
111        min_n: int = 5,
112    ) -> Any:
113        """Whether the owned model is **served-safe** to promote toward default ? wraps
114        :func:`backend.eval.promotion.promote_if_served_safe` (the foundational *no-
regression-at-served-
115        windowing* gate). ``served_`` are per-video served F1s on the same corpus; Gemini
is the control
116        (A), the owned model the proposal (B)."""
117        from backend.eval.promotion import promote_if_served_safe
118
119        return promote_if_served_safe(served_gemini, served_owned, min_n=min_n)
120
121
122    def run_for_job(
123        job: Dict[str, Any],
124        result: Dict[str, Any],
125        config: Any,
126        *,
127        source_path: Optional[str] = None,
128    ) -> Optional[Dict[str, Any]]:
129        """The gated orchestrator called at job completion. Returns ``None`` (a no-op)
unless **both**
130        ``config.flywheel.enabled`` AND :func:`consent_attested` ? so today it never
captures (consent is
131        faked-deny). On a consented job it captures the artifacts into the per-video
workspace; shadow-eval
132        + promotion are wired but stay inert until the owned model + golden labels exist.
Best-effort: the
133        caller wraps this so a bookkeeping hiccup never wedges a completed job."""
134        fw = getattr(config, "flywheel", None)
135        if not fw or not getattr(fw, "enabled", False):
136            return None
137        if not consent_attested(job, config):
138            logger.info("[FLYWHEEL] job %s not captured ? no attested consent (faked-
deny).", job.get("id"))
139            return None
140
141        video_id = (result or {}).get("video_id")
142        if not video_id:
143            logger.info("[FLYWHEEL] job %s has no video_id ? nothing to capture.",
job.get("id"))
144            return None
145
146        storage_cfg = getattr(config, "storage", None)
147        output_dir = getattr(getattr(config, "output", None), "output_dir", "output")
148        root = _workspace.resolve_workspace_root(
149            getattr(storage_cfg, "workspace_root", "") or "",

```

```

150         getattr(storage_cfg, "output_root", "") or "",
151         output_dir,
152     )
153     ws = _workspace.for_video(root, video_id, prefix=fw.workspace_prefix)
154     written = capture_job(
155         ws,
156         rally_output={"play_segments": (result or {}).get("play_segments", [])},
157         source_path=source_path,
158         storage_config=storage_cfg,
159     )
160     logger.info("[FLYWHEEL] captured job %s (video %s) ? %s", job.get("id"), video_id,
written)
161     return {"captured": written, "video_id": video_id}
162

```

6. user (2026-06-22T16:12:22.154Z)

```

1     """M11 data-flywheel harness (COMPUTE_DECOUPLED_SERVING M11, D12).
2
3     The cardinal invariants: **DEFAULT-OFF** (`flywheel.enabled=False` ? strict no-op) and
the
4     **faked consent hard-deny** (`consent_attested` returns False for every job ? nothing
is captured
5     even when enabled). Plus: the capture/shadow-eval/promote wrappers REUSE the real
workspace / eval /
6     promotion code (delegation verified). Pure + offline ? storage is the local (copy)
backend on tmp_path.
7     """
8     import json
9     import os
10
11     import pytest
12
13     from backend import flywheel
14     from backend.config import AppConfig
15
16
17     def _cfg(*, enabled=False, output_dir="output", prefix="flywheel"):
18         return AppConfig.model_validate({
19             "flywheel": {"enabled": enabled, "workspace_prefix": prefix},
20             "output": {"output_dir": output_dir},
21             "storage": {"backend": "local"},
22         })
23
24
25     # ----- default-OFF + consent
guardrail
26
27     def test_flywheel_config_defaults_off():
28         cfg = AppConfig()
29         assert cfg.flywheel.enabled is False
30         assert cfg.flywheel.workspace_prefix == "flywheel"
31
32
33     def test_consent_is_faked_hard_deny():
34         """The privacy guardrail: consent_attested denies EVERY job until counsel + the
schema land."""
35         assert flywheel.consent_attested({"id": 1}, _cfg(enabled=True)) is False
36         assert flywheel.consent_attested({"id": 2, "owner_id": "u@x"}, None) is False
37
38
39     def test_run_for_job_noop_when_disabled():
40         cfg = _cfg(enabled=False)
41         assert flywheel.run_for_job({"id": 1}, {"video_id": "v"}, cfg) is None
42

```



```

43
44     def test_run_for_job_noop_when_enabled_but_no_consent(tmp_path):
45         """enabled=True but consent denied (the faked default) ? still a no-op, NOTHING
written."""
46         cfg = _cfg(enabled=True, output_dir=str(tmp_path))
47         assert flywheel.run_for_job({"id": 1}, {"video_id": "v"}, cfg) is None
48         assert not list(tmp_path.rglob("capture.json"))
49
50
51     # ----- capture
(consigned)
52
53     def test_run_for_job_captures_when_enabled_and_consigned(tmp_path, monkeypatch):
54         """With consent monkeypatched True, a per-video manifest lands under
<output>/flywheel/<vid>/."""
55         monkeypatch.setattr(flywheel, "consent_attested", lambda job, config=None: True)
56         cfg = _cfg(enabled=True, output_dir=str(tmp_path))
57         out = flywheel.run_for_job(
58             {"id": 7}, {"video_id": "abc123", "play_segments": [{"id": 1, "start_time":
0}], cfg)
59         assert out and out["video_id"] == "abc123"
60         manifest = tmp_path / "flywheel" / "abc123" / "outputs" / "capture.json"
61         assert manifest.exists()
62         data = json.loads(manifest.read_text())
63         assert data["video_id"] == "abc123"
64         assert data["rally_output"]["play_segments"][0]["id"] == 1
65
66
67     def test_run_for_job_no_video_id_is_noop(tmp_path, monkeypatch):
68         monkeypatch.setattr(flywheel, "consent_attested", lambda job, config=None: True)
69         cfg = _cfg(enabled=True, output_dir=str(tmp_path))
70         assert flywheel.run_for_job({"id": 7}, {}, cfg) is None
71
72
73     def test_capture_job_persists_manifest_and_source(tmp_path):
74         """capture_job writes the manifest AND (when given) the source video, via
storage.put."""
75         ws = flywheel._workspace.for_video(str(tmp_path), "vid9", prefix="flywheel")
76         src = tmp_path / "in.mp4"
77         src.write_bytes(b"FAKE-MP4")
78         written = flywheel.capture_job(
79             ws, rally_output={"play_segments": []}, source_path=str(src),
80             storage_config=AppConfig().storage)
81         assert os.path.exists(written["manifest_uri"])
82         assert os.path.exists(written["source_uri"])
83         assert (tmp_path / "flywheel" / "vid9" / "source" / "in.mp4").read_bytes() ==
b"FAKE-MP4"
84
85
86     # ----- shadow_eval / promote
delegation
87
88     def test_shadow_eval_labeled_delegates_to_compare(monkeypatch):
89         calls = {}
90         from backend.eval import experiment
91
92         def fake_compare(c, a, b, iou=0.5):
93             calls["compare"] = (c, a, b, iou)
94             return {"ok": 1}
95
96         monkeypatch.setattr(experiment, "compare", fake_compare)
97         res = flywheel.shadow_eval(["c1", "c2"], "owned", "gemini", labeled=True, iou=0.4)
98         assert res == {"ok": 1}
99         assert calls["compare"] == ([ "c1", "c2"], "owned", "gemini", 0.4)
100

```

```

101
102 def test_shadow_eval_unlabeled_delegates_to_compare_unlabeled(monkeypatch):
103     from backend.eval import experiment
104     seen = {}
105
106     def fake_unlabeled(v, a, b, agree_iou=0.5):
107         seen["u"] = (v, a, b, agree_iou)
108         return {"agree": 1}
109
110     monkeypatch.setattr(experiment, "compare_unlabeled", fake_unlabeled)
111     assert flywheel.shadow_eval(["only"], "owned", "gemini") == {"agree": 1}
112     assert seen["u"] == ("only", "owned", "gemini", 0.5)
113
114
115 def test_shadow_eval_unlabeled_requires_single_video():
116     with pytest.raises(ValueError):
117         flywheel.shadow_eval(["a", "b"], "owned", "gemini", labeled=False)
118
119
120 def test_propose_promotion_delegates_with_gemini_as_control(monkeypatch):
121     """The owned model is the PROPOSAL (B); Gemini is the CONTROL (A) ? promote only if
served-safe."""
122     import backend.eval.promotion as promotion
123     seen = {}
124
125     def fake_gate(a, b, *, min_n=5):
126         seen["p"] = (list(a), list(b), min_n)
127         return "VERDICT"
128
129     monkeypatch.setattr(promotion, "promote_if_served_safe", fake_gate)
130     out = flywheel.propose_promotion([0.8, 0.9], [0.7, 0.75], min_n=6) # owned, gemini
131     assert out == "VERDICT"
132     assert seen["p"] == ([0.7, 0.75], [0.8, 0.9], 6) # A=gemini(control),
B=owned(proposal)
133
134
135     # ----- the job-worker
hook (gated)
136
137 def test_job_worker_hook_noop_when_disabled(monkeypatch):
138     """_maybe_run_flywheel must not call into the harness when flywheel is disabled."""
139     import backend.main as bmain
140     from backend.api import job_worker
141
142     monkeypatch.setattr(bmain, "global_config", _cfg(enabled=False), raising=False)
143     called = []
144     monkeypatch.setattr(flywheel, "run_for_job", lambda *a, **k: called.append(1))
145     job_worker._maybe_run_flywheel({"id": 1}, {"video_id": "v"})
146     assert called == []
147
148
149 def test_job_worker_hook_is_best_effort(monkeypatch):
150     """An exception inside the flywheel never propagates (a completed job must not be
wedged)."""
151     import backend.main as bmain
152     from backend.api import job_worker
153
154     monkeypatch.setattr(bmain, "global_config", _cfg(enabled=True), raising=False)
155
156     def _boom(*a, **k):
157         raise RuntimeError("kaboom")
158
159     monkeypatch.setattr(flywheel, "run_for_job", _boom)
160     job_worker._maybe_run_flywheel({"id": 1}, {"video_id": "v"}) # must NOT raise
161

```

7. user (2026-06-22T16:12:23.449Z)

```
1      """Promotion gate with the eval?serve guard: a cue is promotable ONLY if it does not
regress
2      at the SERVED windowing.
3
4      **Why this exists.** ``per_user_gate.should_promote_for_user`` already grades a B?A lift
5      honestly (min-N floor + ?F1 CI excludes 0 + paired sign test). But it grades whatever
scores
6      it is handed ? and the foundational bug was that those scores came from the *recall-
oriented
7      PROPOSAL* windowing (``backend.eval.windowing.PROPOSAL``), which production never
serves. So a
8      cue could clear the bar on a windowing nobody runs (Gate-A: proposal ?F1 **+0.074**)
while
9      regressing on the served one (**?0.008**). Grading harder isn't the fix; **measuring on
the
10     served windowing** is.
11
12     This module composes ? it adds no new statistics. It layers ONE rule on top of
13     ``should_promote_for_user``:
14
15     A cue is promotable only if, at the **SERVED** windowing, it WINS or at least DOES
NOT
16     REGRESS (its served paired ?F1 mean ? ``-served_regress_eps``, and ? when its served
CI is
17     estimable ? that CI's lower bound does not sit clearly in regression territory).
18
19     A proposal-windowing win is still *reported* (it's useful signal for where to look
next), but it
20     can never carry a promotion on its own. The decision returns BOTH the served and the
proposal
21     verdicts so the eval?serve contrast is explicit and auditable.
22
23     Pure, deterministic, stdlib-only over ``eval_stats``/``per_user_gate``. Additive:
existing eval
24     paths are untouched until a caller routes its A/B through
:func:`promote_if_served_safe`.
25     """
26     from __future__ import annotations
27
28     from dataclasses import dataclass, field
29     from typing import Any, Dict, Optional, Sequence
30
31     from backend.eval.eval_stats import paired_delta_ci
32     from backend.eval.per_user_gate import PromotionDecision, should_promote_for_user
33
34     __all__ = ["ServedGateResult", "promote_if_served_safe", "served_regresses"]
35
36
37     def served_regresses(a_scores: Sequence[float], b_scores: Sequence[float], *,
38                         eps: float = 0.0, n_boot: int = 2000, seed: int = 0) -> Dict[str,
Any]:
39         """Does variant B REGRESS variant A at the served windowing? (the eval?serve guard
core).
40
41         ``a_scores``/``b_scores`` are per-video held-out scores **measured at the SERVED
windowing**
42         (aligned by video). Returns the paired B?A summary (mean ?F1 + bootstrap CI + sign
test) plus
43         a single ``regresses`` boolean:
44
45         - ``regresses=True`` when the served mean ?F1 is below ``-eps`` (a real served
drop), OR
46         the served ?F1 CI lies **entirely** below ``-eps`` (a confidently-negative
```

```

band).
47         - ``regresses=False`` when the cue holds the line at the served operating point
(mean ?
48         ``-eps`` and the CI is not a confident net-negative).
49
50         ``eps`` is the regression tolerance: 0.0 = "must not drop at all on the served
mean". A tiny
51         positive ``eps`` lets noise-level wobble through (e.g. 0.005 ? half an F1 point at
n?6).
52         ""
53         delta = paired_delta_ci(a_scores, b_scores, n_boot=n_boot, seed=seed)
54         mean_delta = float(delta["mean_delta"])
55         ci_low = float(delta["ci_low"])
56         ci_high = float(delta["ci_high"])
57         # Regression if the served mean dropped beyond tolerance, OR the whole CI is net-
negative.
58         mean_regresses = mean_delta < -eps
59         ci_confidently_negative = ci_high < -eps
60         regresses = bool(mean_regresses or ci_confidently_negative)
61         return {
62             "regresses": regresses,
63             "mean_delta": mean_delta,
64             "ci_low": ci_low,
65             "ci_high": ci_high,
66             "ci_excludes_zero": bool(delta["ci_excludes_zero"]),
67             "sign_test": delta["sign_test"],
68             "n": int(delta["n"]),
69             "eps": float(eps),
70         }
71
72
73     @dataclass
74     class ServedGateResult:
75         """A promotion decision that BOTH measures the cue on the served windowing AND
reports the
76         proposal-windowing view, so the eval?serve contrast is never hidden.
77
78         - ``promote`` ? final verdict. ``True`` only when the underlying honest gate would
promote
79         *and* the served check did not veto (the cue does not regress at the served
windowing).
80         - ``served_safe`` ? the eval?serve guard's verdict in isolation (cue does not
regress served).
81         - ``vetoed_by_served`` ? ``True`` when the proposal/base gate WOULD have promoted
but the
82         served regression check overruled it (the exact Gate-A failure mode this module
prevents).
83         - ``base_decision`` ? the ``PromotionDecision`` from ``should_promote_for_user`` on
the
84         *promotion-grade* scores (served by default ? see :func:`promote_if_served_safe`).
85         - ``served`` / ``proposal`` ? the paired B?A summary at each windowing (means + CIs
+ sign
86         tests), so a reviewer sees "+0.074 proposal / ?0.008 served" side by side.
87         - ``reason`` ? short human-readable explanation of the final verdict.
88         ""
89
90         promote: bool
91         served_safe: bool
92         vetoed_by_served: bool
93         reason: str
94         base_decision: PromotionDecision
95         served: Dict[str, Any]
96         proposal: Optional[Dict[str, Any]] = field(default=None)
97
98         def as_dict(self) -> Dict[str, Any]:

```

```

99         return {
100             "promote": self.promote,
101             "served_safe": self.served_safe,
102             "vetoed_by_served": self.vetoed_by_served,
103             "reason": self.reason,
104             "base_decision": self.base_decision.as_dict(),
105             "served": self.served,
106             "proposal": self.proposal,
107         }
108
109
110     def promote_if_served_safe(
111         served_a: Sequence[float],
112         served_b: Sequence[float],
113         *,
114         proposal_a: Optional[Sequence[float]] = None,
115         proposal_b: Optional[Sequence[float]] = None,
116         structural: bool = False,
117         min_n: int = 5,
118         p_threshold: float = 0.05,
119         served_regress_eps: float = 0.0,
120         n_boot: int = 2000,
121         seed: int = 0,
122     ) -> ServedGateResult:
123         """Promote a cue ONLY if it does not regress at the SERVED windowing (the
124         foundational rule).
125
126         The honest bar (`should_promote_for_user`: min-N floor + ?F1 CI excludes 0 + sign
127         test) is
128         applied to the **served** scores ? the operating point production runs ? so a
129         proposal-only
130         win can never carry a promotion. The optional `proposal_` scores are scored too
131         and
132         reported alongside, purely to surface the eval?serve contrast (they NEVER promote on
133         their
134         own).
135
136         Args:
137             served_a/served_b: per-video held-out scores for variant A (control) and B (the
138             cue),
139             measured at the **SERVED** windowing. These drive the decision.
140             proposal_a/proposal_b: the same scores at the PROPOSAL windowing (optional).
141         Reported for
142             contrast; the served regression guard still has the final say.
143             structural: forwarded to `should_promote_for_user` ? a structural guard (e.g.
144             Gate-A vs
145             held-shuttle FPs) is never *disabled* on "no lift", but it is ALSO not
146             auto-*promoted* to
147             the served default if it regresses served. So for `structural=True` we report
148             `keep_on` honestly but `promote` reflects the served-safety check.
149             served_regress_eps: served regression tolerance (0.0 = "no served-mean drop
150             allowed").
151
152         Returns a :class:`ServedGateResult` with the final verdict + both windowings'
153         evidence.
154
155         """
156         # 1) The honest gate, applied to the SERVED scores (never the proposal ones).
157         base = should_promote_for_user(
158             served_a, served_b,
159             structural=structural, min_n=min_n, p_threshold=p_threshold,
160             n_boot=n_boot, seed=seed,
161         )
162
163         # 2) The eval?serve guard: does B regress A at the served windowing?
164         served = served_regresses(served_a, served_b, eps=served_regress_eps,

```

```

153             n_boot=n_boot, seed=seed)
154     served_safe = not served["regresses"]
155
156     # 3) Optional proposal-windowing view (reported, never promoting on its own).
157     proposal: Optional[Dict[str, Any]] = None
158     if proposal_a is not None and proposal_b is not None:
159         proposal = served_regresses(proposal_a, proposal_b, eps=served_regress_eps,
160                                     n_boot=n_boot, seed=seed)
161
162     # 4) Compose the verdict. The served guard can only ever REMOVE a promotion, never
163     add one.
164     if structural:
165         # A structural guard ships ON regardless of measured lift (base.keep_on stays
166         True), but
167         # it is only auto-promoted to the SERVED default when it is served-safe. If it
168         regresses
169         # served, promote=False (don't bake an eval-only win into production) while
170         keep_on holds.
171         promote = served_safe
172         vetoed = False
173         if served_safe:
174             reason = ("structural guard kept ON; served-safe (does not regress at the
175             served "
176                     f"windowing, ?F1 {served['mean_delta']:+.4f}) ? eligible for
177             served default")
178         else:
179             reason = ("structural guard kept ON, but NOT promoted to the served default:
180             it "
181                     f"REGRESSES at the served windowing (?F1
182             {served['mean_delta']:+.4f}); "
183                     "an eval-only win must not be served")
184         else:
185             would_promote = base.promote
186             if would_promote and not served_safe:
187                 promote = False
188                 vetoed = True
189                 reason = ("VETOED by the served-windowing guard: the cue clears the honest
190             bar but "
191                     f"REGRESSES at the served windowing (served ?F1
192             {served['mean_delta']:+.4f})"
193                     f", CI [{served['ci_low']:+.4f}, {served['ci_high']:+.4f}]). A win
194             on a "
195                     "windowing production doesn't serve cannot promote.")
196             elif would_promote:
197                 promote = True
198                 vetoed = False
199                 reason = ("promoted: clears the honest bar AND is served-safe (served ?F1 "
200             f"{served['mean_delta']:+.4f}). " + base.reason)
201             else:
202                 promote = False
203                 vetoed = False
204                 reason = base.reason
205
206     return ServedGateResult(
207         promote=promote,
208         served_safe=served_safe,
209         vetoed_by_served=vetoed,
210         reason=reason,
211         base_decision=base,
212         served=served,
213         proposal=proposal,
214     )

```

```

1  """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3  This is the thin variant/experiment layer that `docs/EXPERIMENT_HARNESS.md`
4  identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5  no-regression gate, the pinnable classifier artifact, the persisted-candidate
6  substrate) already exists ? this module composes it. No new scoring math lives
7  here*; everything defers to:
8
9      - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10     - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11     - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12     - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13     - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14     - `backend.eval.calibration`    ? `pct_improvement`
15
16  The thesis (`EXPERIMENT_HARNESS.md` §2): separate the expensive, risky DETECTION
17  (per-frame signals ? run once on the GPU/WSL box, persisted into
18  `candidate_segments.stats`) from the cheap, swappable DECISION (given those
19  signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20  given persisted candidate features it produces `List[Interval]` deterministically,
21  with no GPU and zero production risk. So most experiments are pure offline
22  re-scoring of stored data and never touch the served pipeline.
23
24  Three modes (`EXPERIMENT_HARNESS.md` §5):
25      - `compare()`      ? labeled A/B vs golden labels: per-video + LOVO-honest
26        aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27        held-out loop, but variant-parameterised.
28      - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29        variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30        human spot-checks (and what two highlight reels would show side by side).
31      - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32        (exit 0/1/3); rollback = flip the variant/config, never `git revert`.
33
34  Pure + offline + unit-tested. The only DB-touching helper is
35  `load_video_candidates`, which reads persisted candidates via
36  `Database.query_candidate_segments`.
37  """
38  from __future__ import annotations
39
40  import glob
41  import json
42  import logging
43  import os
44  from dataclasses import dataclass, field
45  from datetime import datetime, timezone
46  from hashlib import sha1
47  from typing import Any, Callable, Dict, List, Optional, Sequence
48
49  from backend.eval.calibration import pct_improvement
50  from backend.eval.classifier import LogisticRegression
51  from backend.eval.gemini_refine import merge_overlaps
52  from backend.eval.metrics import Interval, match_intervals, score
53  from backend.eval.regression import gt_hash
54  from backend.eval.training import label_candidates
55  from backend.eval import eval_stats as _stats          # C1: CIs + paired sign test
56  from backend.eval import segmentation_metrics as _segm  # C4: F1@k + segment-count
57  ratio
58  from backend.eval.score_calibration import fit_isotonic, fit_platt  # C2: calibrate cue
59  scores
60
61  logger = logging.getLogger(__name__)
62
63  # Where a comparison run appends one reproducible record. Tracked location (NOT under
64  # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
65  # regression.DEFAULT_BASELINE_PATH.

```

```

64     DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
65     _METRICS = ("precision", "recall", "f1", "mean_iou")
66
67
68     class ExperimentError(ValueError):
69         """A variant cannot be evaluated as configured (e.g. a learned variant asked to
70         score an unlabeled video with no pinned model, or a gate referencing an absent
71         feature)."""
72
73
74     # ----- Variant
75
76
77     @dataclass
78     class Variant:
79         """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
80
81         A variant turns one video's persisted candidate windows + features into rally
82         intervals. Every knob defaults to the control behaviour (no gate, no learned
83         filter), so a variant that merely carries a new, disabled cue is byte-identical
84         to control ? the core no-regression guarantee.
85
86         Fields:
87         - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
88           for the leaderboard and for selecting the served path (the existing
89           ``segmenter_registry`` + ``IndexingConfig`` do the actual selection in production).
90           New cues are recorded here default-OFF.
91         - ``min_crossings`` ? decision-gate Gate A: keep only windows whose persisted
92           ``net_crossings`` feature is  $\geq$  this. 0 = off. This is the eval-only gate from
93           ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
shuttle
94           windows ? see ``MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
95         - ``min_players`` ? decision-gate Gate B (the future person cue): keep windows
96           whose persisted ``players_in_court`` is  $\geq$  this. 0 = off (no-op until the person
97           cue persists that feature).
98         - ``classifier_model`` ? path to a frozen ``LogisticRegression`` JSON. When set,
99           ``compare()`` scores videos with the SHIPPED model as-is (no per-fold training);
100           required for ``compare_unlabeled`` on a learned variant.
101         - ``feature_names`` ? the persisted features the learned filter consumes. When set
102           WITHOUT ``classifier_model``, ``compare()`` trains a fresh model per LOVO fold
103           (honest) ? the recipe, not a pinned artifact.
104         - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
105           overlap-merge gap (seconds), the same ``gemini_refine.merge_overlaps`` default.
106         """
107
108         name: str
109         segmenter: str = "wasb_hybrid"
110         config_overrides: Dict[str, Any] = field(default_factory=dict)
111         cue_flags: Dict[str, bool] = field(default_factory=dict)
112         min_crossings: int = 0
113         min_players: int = 0
114         classifier_model: Optional[str] = None
115         feature_names: Optional[List[str]] = None
116         threshold: float = 0.5
117         merge_gap: float = 1.0
118         # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
119         # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
120         # failure where a fixed 0.5 zeroed out a small-candidate video.
121         tune_threshold: bool = False # pick the F1-best threshold on the train fold
122         calibrate: Optional[str] = None # None | "platt" | "isotonic" ? calibrate
proba first
123
124         def is_learned(self) -> bool:

```



```

125         """True if this variant applies a learned classifier (pinned model or
recipe)."""
126         return self.classifier_model is not None or bool(self.feature_names)
127
128     def to_dict(self) -> dict:
129         return {
130             "name": self.name,
131             "segmenter": self.segmenter,
132             "config_overrides": self.config_overrides,
133             "cue_flags": self.cue_flags,
134             "min_crossings": self.min_crossings,
135             "min_players": self.min_players,
136             "classifier_model": self.classifier_model,
137             "feature_names": self.feature_names,
138             "threshold": self.threshold,
139             "merge_gap": self.merge_gap,
140             "tune_threshold": self.tune_threshold,
141             "calibrate": self.calibrate,
142         }
143
144     @classmethod
145     def from_dict(cls, d: dict) -> "Variant":
146         known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
147         return cls(**{k: v for k, v in d.items() if k in known})
148
149     def spec_hash(self) -> str:
150         """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
151         and makes a result reproducible. The pinned **control** variant always hashes
152         the
153         same, so a regression is always traceable to a recipe change, never to drift."""
154         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
155         return sha1(blob.encode()).hexdigest()[:12]
156
157     #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
158     #: filter. Always the comparison reference; "rollback" = make this the served variant.
159     CONTROL = Variant(name="control")
160
161
162     # ----- persisted candidates
163
164
165     @dataclass
166     class VideoCandidates:
167         """One video's persisted detection, ready for offline re-scoring.
168
169         ``intervals`` are the candidate windows; ``features`` is column-major
170         (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
171         ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
172         with ``load_video_candidates``, or directly in tests.
173         """
174
175         name: str
176         intervals: List[Interval]
177         features: Dict[str, List[float]] = field(default_factory=dict)
178         gts: Optional[List[Interval]] = None
179
180
181     def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
182         """True iff every candidate row carries the per-cue feature dicts the golden-feature
loaders
183         read (``shuttle`` + ``person``; ``audio`` stays optional ? it is added later by
184         ``fusion_audio.augment`` and older JSONs predate it).
185
186         The velocity-windowing / distill corpus-growth path persists a LEANER

```

```

187     ``*_golden_features.json`` row (``start``/``end`` + velocity stats, no cue dicts)
for clips
188     that never went through fusion feature extraction. The loaders that build cue-rich
189     ``VideoCandidates`` from these files ? ``ablation.load_videos``,
``fusion_compare.load_videos``
190     and ``fusion_audio.load_videos_with_audio`` ? SKIP those rather than crash on
``c["shuttle"]``,
191     so a MIXED corpus (some fusion files, some windowing-only) loads the fusion subset
cleanly.
192     An empty candidate list is not a fusion file either.
193
194     Single source of truth for the three loaders (consolidated 2026-06-18 after PRs
#209/#210
195     each landed an independent copy)."""
196     return bool(candidates) and all("shuttle" in c and "person" in c for c in
candidates)
197
198
199     def load_golden_videos(
200         features_dir: str,
201         *,
202         shuttle_columns: Sequence[str],
203         person_columns: Sequence[str],
204         audio_columns: Sequence[str] = (),
205         audio_policy: str = "omit",
206         skip_logger: logging.Logger = logger,
207         skip_message: str,
208         no_audio_error: str = "{path} has no audio features",
209     ) -> List["VideoCandidates"]:
210         """Shared golden-feature corpus loader behind the three structurally-identical
loaders
211         (``ablation.load_videos`` / ``fusion_compare.load_videos`` /
``fusion_audio.load_videos_with_audio``).
212
213         Reads every FULL-FUSION-SCHEMA ``*_golden_features.json`` in ``features_dir`` into a
214         ``VideoCandidates`` (intervals + the caller-supplied feature columns + golden gts).
Leaner
215         velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the schema the
216         corpus-growth path persists) are SKIPPED, not crashed on (see
``_is_fusion_schema``), so a
217         MIXED corpus loads the fusion subset cleanly.
218
219         The column lists are ALWAYS supplied by the caller ? this loader never hardcodes
columns. The
220         three callers differ ONLY in:
221
222         - ``audio_policy`` ? how the audio block is handled, checked BEFORE reading any
audio column
223         so a windowing-only file is skipped before either branch applies:
224         * ``"omit"`` ? no audio columns are loaded (``audio_columns``
ignored);
225         * ``"tolerate-missing"`` ? audio columns load, a missing block defaults them
to 0.0;
226         * ``"require"`` ? a fusion file lacking audio raises ``SystemExit``
(via
227         ``no_audio_error`` formatted with ``path``), mirroring "run ``--augment``
first".
228         - ``skip_logger`` / ``skip_message`` ? the per-caller logger + ``%s``-templated
info line
229         emitted when a windowing-only file is skipped (kept byte-identical per caller).
230         """
231         if audio_policy not in ("omit", "tolerate-missing", "require"):
232             raise ValueError(
233                 f"audio_policy must be 'omit'/'tolerate-missing'/'require', got
{audio_policy!r}")

```

```

234     videos: List[VideoCandidates] = []
235     for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
236         with open(path, encoding="utf-8") as f:
237             d = json.load(f)
238             cands = d["candidates"]
239             if not _is_fusion_schema(cands):
240                 skip_logger.info(skip_message, os.path.basename(path))
241                 continue
242             if audio_policy == "require" and "audio" not in cands[0]:
243                 raise SystemExit(no_audio_error.format(path=path))
244             intervals = [(float(c["start"]), float(c["end"])) for c in cands]
245             features: Dict[str, List[float]] = {}
246             for fn in shuttle_columns:
247                 features[fn] = [float(c["shuttle"][fn]) for c in cands]
248             for fn in person_columns:
249                 features[fn] = [float(c["person"][fn]) for c in cands]
250             if audio_policy == "tolerate-missing":
251                 for fn in audio_columns:
252                     features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in
cands]
253             elif audio_policy == "require":
254                 for fn in audio_columns:
255                     features[fn] = [float(c["audio"][fn]) for c in cands]
256             gts = [(float(a), float(b)) for a, b in d["gts"]]
257             videos.append(VideoCandidates(name=d["name"], intervals=intervals,
258                                         features=features, gts=gts))
259     return videos
260
261
262     def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
263         """Persisted feature column, defaulting missing/short columns to 0.0 (matches
264         `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
265         col = vc.features.get(name)
266         n = len(vc.intervals)
267         if col is None:
268             logger.warning("variant references feature %r absent from %s ? treating as 0.0",
269                             name, vc.name)
270             return [0.0] * n
271         if len(col) != n:
272             raise ExperimentError(
273                 f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
274         return [float(x) for x in col]
275
276
277     def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
278         cols = [_feature_column(vc, name) for name in feature_names]
279         return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
280
281
282     def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
283         """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
284         players)."""
285         n = len(vc.intervals)
286         mask = [True] * n
287         if variant.min_crossings > 0:
288             col = _feature_column(vc, "net_crossings")
289             mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
290         if variant.min_players > 0:
291             col = _feature_column(vc, "players_in_court")
292             mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
293         return mask
294
295     def predict(variant: Variant, vc: VideoCandidates,

```

```

296         model: Optional[LogisticRegression] = None,
297         threshold: Optional[float] = None,
298         calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
299     """Variant prediction: gate by persisted features, optionally filter by a learned
300     model, then merge. Pure and deterministic.
301
302     ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
303     fold; if omitted and the variant pins ``classifier_model``, that frozen model is
304     loaded. A learned variant with neither a supplied nor a pinned model raises ? there
305     is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
306     fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
307     them.
308     """
309     mask = _gate_mask(variant, vc)
310     if variant.feature_names:
311         if model is None:
312             if variant.classifier_model is None:
313                 raise ExperimentError(
314                     f"variant {variant.name!r} is learned but has no model to predict "
315                     f"with (pass a fitted model or set classifier_model)")
316             model = LogisticRegression.load(variant.classifier_model)
317             proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
318 variant.feature_names))]
319             if calibrator is not None:
320                 proba = [calibrator(p) for p in proba]
321                 thr = variant.threshold if threshold is None else threshold
322                 mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
323             kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
324             return merge_overlaps(kept, gap_tol=variant.merge_gap)
325
326     def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
327                             train_videos: Sequence[VideoCandidates], iou: float
328                             ) -> "tuple[Optional[Callable[[float], float]],
329 Optional[float]]":
330         """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
331         (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
332         (use the variant default). Honest: never looks at the held-out video.
333         """
334         calibrator: Optional[Callable[[float], float]] = None
335         if variant.calibrate:
336             raw: List[float] = []
337             y: List[int] = []
338             for vc in train_videos:
339                 raw.extend(float(p) for p in
340                             model.predict_proba(_feature_matrix(vc, variant.feature_names or
341 [])))
342                 y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
343             if variant.calibrate == "platt":
344                 calibrator = fit_platt(raw, y)
345             elif variant.calibrate == "isotonic":
346                 calibrator = fit_isotonic(raw, y)
347             else:
348                 raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
349 'platt'/'isotonic')")
350             threshold: Optional[float] = None
351             if variant.tune_threshold:
352                 best_t, best_f1 = variant.threshold, -1.0
353                 for k in range(1, 20):
354                     # grid 0.05..0.95 on the train
355                     fold's rally F1
356                     t = round(0.05 * k, 2)
357                     f1s = [score(predict(variant, vc, model=model, threshold=t,
358 calibrator=calibrator),
359                               vc.gts or [], iou).f1 for vc in train_videos]
360                     mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0

```

```

354         if mean_f1 > best_f1:
355             best_f1, best_t = mean_f1, t
356         threshold = best_t
357     return calibrator, threshold
358
359
360     # ----- variant scoring
361
362
363     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
364         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
365         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
366         X: List[List[float]] = []
367         y: List[int] = []
368         for vc in videos:
369             if vc.gts is None:
370                 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
371             X.extend(_feature_matrix(vc, variant.feature_names or []))
372             y.extend(label_candidates(vc.intervals, vc.gts, iou))
373         if not X:
374             raise ExperimentError("cannot train a learned variant on zero candidates")
375         return LogisticRegression().fit(X, y, variant.feature_names)
376
377
378     def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
379                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
380         """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
381         mean aggregate.
382
383         Prediction policy:
384         - **static variant** (no learned filter): predict each video directly ? no
training,
385         so no leakage; per-video scoring is already honest.
386         - **learned, pinned model** (``classifier_model`` set): score every video with the
387         SHIPPED model. ? optimistic if those videos were in its training set ? use this
to
388         report "how the shipped artifact does here", not for the promotion decision.
389         - **learned recipe** (``feature_names``, no pinned model) + ``honest=True``: LOVO
?
390         train on the OTHER videos, predict the held-out one. The number to trust.
391         - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity
only).
392         """
393         for vc in videos:
394             if vc.gts is None:
395                 raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
396
397         per_video: List[Dict[str, Any]] = []
398         predictions: Dict[str, List[Interval]] = {}
399
400         recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
401         pinned_model = (LogisticRegression.load(variant.classifier_model)
402                         if variant.classifier_model is not None else None)
403         all_model = None
404         if recipe_lovo and not honest:
405             all_model = _train(variant, videos, iou)
406
407         for i, vc in enumerate(videos):
408             cal: Optional[Callable[[float], float]] = None
409             thr: Optional[float] = None
410             if recipe_lovo and honest:
411                 others = [v for j, v in enumerate(videos) if j != i]

```

```

412         if not others:
413             raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
414         model: Optional[LogisticRegression] = _train(variant, others, iou)
415         if variant.tune_threshold or variant.calibrate: # operating point from
the train fold
416             assert model is not None # _train never returns
None
417             cal, thr = _calibrate_and_tune(variant, model, others, iou)
418         elif recipe_lovo: # honest=False ? one model over all
videos
419             model = all_model
420         else: # static variant or pinned model
421             model = pinned_model
422         preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
423         assert vc.gts is not None # guarded at function top
424         sc = score(preds, vc.gts, iou)
425         per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
426         predictions[vc.name] = preds
427
428         aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
429                     for m in _METRICS}
430     return {
431         "variant": variant.name,
432         "spec_hash": variant.spec_hash(),
433         "is_learned": variant.is_learned(),
434         "honest_lovo": recipe_lovo and honest,
435         "per_video": per_video,
436         "aggregate": aggregate,
437         "predictions": predictions,
438     }
439
440
441     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
442         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
443         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}
444
445
446     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
447         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
448         gts_by_name = {v.name: (v.gts or []) for v in videos}
449         overlaps = _segm.DEFAULT_OVERLAPS
450         f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
451         ratios: List[float] = []
452         for name, preds in eval_v.get("predictions", {}).items():
453             gts = gts_by_name.get(name, [])
454             got = _segm.f1_at_overlaps(preds, gts, overlaps)
455             for ov in overlaps:
456                 f1k[ov].append(got[ov])
457             ratios.append(_segm.segment_count_ratio(preds, gts))
458
459         def _mean(xs: List[float]) -> float:
460             return sum(xs) / len(xs) if xs else 0.0
461         out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
462         out["segment_count_ratio"] = _mean(ratios)
463         return out
464
465

```

```

466     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
467                        eval_b: Dict[str, Any]) -> Dict[str, Any]:
468         """C1 + C4 honest reporting layered over a compare() pair (additive,
469         EXPERIMENT_HARNESS §6).
470         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
471         order),
472         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
473         when
474         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
475         """
476         a_f1 = [p["f1"] for p in eval_a["per_video"]]
477         b_f1 = [p["f1"] for p in eval_b["per_video"]]
478         return {
479             "variant_a_ci": _ci_block(eval_a),
480             "variant_b_ci": _ci_block(eval_b),
481             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
482             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
483                             "variant_b": _segmentation_block(videos, eval_b)},
484         }
485
486     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
487                iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
488     Dict[str, Any]:
489         """Offline labeled A/B (EXPERIMENT_HARNESS.md §5.1). Scores both variants on the
490         same
491         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
492         The deltas use the existing `metrics.score` (per video) +
493         `calibration.pct_improvement`;
494         learned variants are scored LOVO-honestly. The result is a self-contained record fit
495         for
496         `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
497         it.
498         ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
499         CIs, a
500         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
501         *alongside* the
502         existing bare-mean fields (additive: nothing existing changes). Set False for the
503         legacy
504         shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
505         §13/C1+C4): a
506         bare LOVO mean at n?6 is not trustworthy on its own.
507         """
508         if len(videos) < 1:
509             raise ExperimentError("compare needs at least one labeled video")
510         eval_a = evaluate_variant(videos, variant_a, iou, honest)
511         eval_b = evaluate_variant(videos, variant_b, iou, honest)
512
513         delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
514         delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
515         m in _METRICS}
516
517         by_video_a = {p["video"]: p for p in eval_a["per_video"]}
518         per_video_delta = [
519             {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
520             for p in eval_b["per_video"]]
521         ]
522
523         # One fingerprint over the whole label set ? detects "the deltas were measured
524         against
525         # different labels" the same way regression.gt_hash guards the no-regression
526         baseline.

```

```

516 all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
517
518 result: Dict[str, Any] = {
519     "iou": iou,
520     "label_set_hash": gt_hash(all_gts),
521     "num_videos": len(videos),
522     "variant_a": eval_a,
523     "variant_b": eval_b,
524     "delta": delta,
525     "delta_pct": delta_pct,
526     "per_video_delta": per_video_delta,
527 }
528 if honest_stats:
529     result["honest"] = honest_summary(videos, eval_a, eval_b)
530 return result
531
532
533 # ----- SxS on unlabeled video
534
535
536 def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
537     agree_iou: float = 0.5) -> Dict[str, Any]:
538     """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
539
540     With no labels there is no lift to measure ? instead this surfaces where the two
541     variants **agree vs diverge**, which is exactly what a human reviews (and what two
542     highlight reels would show side by side):
543
544     - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
545     - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
546     - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPS:
547       the human / a later golden label adjudicates).
548
549     Both variants must be predictable without labels: a learned variant therefore needs
550     a
551     pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
552     ``metrics.match_intervals`` ? no new matching logic.
553     """
554     preds_a = predict(variant_a, video)
555     preds_b = predict(variant_b, video)
556     mr = match_intervals(preds_a, preds_b, agree_iou)
557
558     agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
559               for m in mr.matches]
560     agree_ious = [m.iou for m in mr.matches]
561     a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
562     b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
563
564     def _dur(ivs: List[Interval]) -> float:
565         return sum(e - s for s, e in ivs)
566
567     return {
568         "video": video.name,
569         "agree_iou": agree_iou,
570         "variant_a": variant_a.name,
571         "variant_b": variant_b.name,
572         "num_a": len(preds_a),
573         "num_b": len(preds_b),
574         "num_agreed": len(agreed),
575         "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
576         "duration_a": _dur(preds_a),
577         "duration_b": _dur(preds_b),
578         "agreed": agreed,
579         "a_only": a_only,
580         "b_only": b_only,

```



```

580     }
581
582
583     # ----- leaderboard / DB
584
585
586     def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
587                           timestamp: Optional[str] = None) -> Dict[str, Any]:
588         """Append a compact, reproducible record of a `compare()` result to the JSON
589         leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
590         `regression_baseline.json`; deliberately the size of a baseline file, not a service
591         (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
592         """
593         row: Dict[str, Any] = {
594             "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
595             "iou": result.get("iou"),
596             "label_set_hash": result.get("label_set_hash"),
597             "num_videos": result.get("num_videos"),
598             "variant_a": {"name": result["variant_a"]["variant"],
599                           "spec_hash": result["variant_a"]["spec_hash"],
600                           "aggregate": result["variant_a"]["aggregate"]},
601             "variant_b": {"name": result["variant_b"]["variant"],
602                           "spec_hash": result["variant_b"]["spec_hash"],
603                           "aggregate": result["variant_b"]["aggregate"]},
604             "delta": result.get("delta"),
605         }
606         # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
607         the
608         # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
609         honest = result.get("honest") or {}
610         if honest.get("paired_delta_f1"):
611             row["honest_delta_f1"] = honest["paired_delta_f1"]
612         entries: List[Dict[str, Any]] = []
613         if os.path.isfile(path):
614             with open(path, encoding="utf-8") as f:
615                 entries = json.load(f)
616         entries.append(row)
617         os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
618         with open(path, "w", encoding="utf-8") as f:
619             json.dump(entries, f, indent=2)
620         logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
621                     row["variant_a"]["name"], row["variant_b"]["name"],
622                     (result.get("delta") or {}).get("f1", 0.0), path)
623         return row
624
625     def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
626                             gts: Optional[List[Interval]] = None,
627                             feature_names: Sequence[str] = ()) -> VideoCandidates:
628         """Bridge persisted detection ? `VideoCandidates` via
629         `Database.query_candidate_segments`.
630
631         ``feature_names`` selects which persisted features to lift out of each row's
632         ``stats``
633         JSON (``"duration"`` is derived from the window bounds). This is the only DB-
634         touching
635         function; the comparison itself is pure and offline. Exercised owner-side on a real
636         DB,
637         but works against any `Database` (including an in-memory one in tests).
638         """
639         rows = db.query_candidate_segments(video_id, detector=detector)
640         intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
641                                     rows]
642         features: Dict[str, List[float]] = {}
643         for name in feature_names:

```

```

639         if name == "duration":
640             features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
641         else:
642             features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
643     return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
644

```

9. assistant (2026-06-22T16:12:27.405Z)

Now let me read the relevant workspace and storage code:

10. user (2026-06-22T16:12:28.916Z)

```

1     """Per-video workspace path helper ? one folder per video, local?cloud identical.
2
3     See ``docs/PER_VIDEO_WORKSPACE.md``. The goal: every per-video artifact lives under
4     ONE folder keyed by the MD5 ``video_id``
5     (``backend/utils/hashing.py::compute_video_id``):
6
6         <root>/[<prefix>]/<video_id>/{source,detector,outputs,reels,logs,tmp}/...
7
8     The ``root`` is a **storage URI** (``local`` path / ``gcs://`` / ``s3://`` /
9     ``gdrive://``), so a
10    LOCAL deployment and a CLOUD (bucket) deployment share the *same relative layout* ? only
11    the root
12    differs. This module is **pure** (string/URI joins, no I/O) so it is trivially testable
13    and works
14    for every backend; the storage layer (``backend/storage``) does the actual read/write.
15
16    Two roots by design (D3): a **durable** workspace root (may be ``gcs://``) for artifacts
17    worth
18    keeping, and a **fast local compute-scratch** mirror (always local) for frame caches /
19    extracted
20    chunks that must never be written to a cloud root. ``tmp`` is the local-only bucket.
21
22    This is Phase 0: the helper + config fields only ? NOT yet wired into write sites (that
23    converges
24    in P2?P4, flag-gated by ``storage.single_folder_per_video``). No behaviour changes on
25    import.
26
27    """
28    from __future__ import annotations
29
30    import os
31    from dataclasses import dataclass
32
33    # Per-video subfolders. ``tmp`` is LOCAL-ONLY (frame PNGs, handoff/yolo/gemini chunks,
34    compile
35    # clips) ? huge + ephemeral, never pushed to a cloud workspace; cache-hygiene drops it.
36    SUBDIRS = ("source", "detector", "outputs", "reels", "logs", "tmp")
37    LOCAL_ONLY_SUBDIRS = ("tmp",)
38
39    def join_uri(root: str, *parts: str) -> str:
40        """Join path components onto a storage root with forward slashes.
41
42        Matches how the codebase already composes storage URIs (e.g.
43        ``f"{prefix}/weights/..."`` in
44        ``backend/worker``). Works for ``gcs://bucket``, ``s3://bucket``, ``local:///abs``
45        and bare
46        paths alike; empty/blank parts are skipped so ``join_uri(root, "", "x")`` ==
47        ``join_uri(root,
48        "x")``. Forward slashes are valid path separators in Python on Windows too.

```

```

38         """
39         base = root.rstrip("/")
40         # Skip empty / whitespace-only parts; for real parts strip only slashes so spaces
inside a
41         # name (e.g. a mounted "My Drive" path) are preserved.
42         tail = "/".join(p.strip("/") for p in parts if p and p.strip())
43         return f"{base}/{tail}" if tail else base
44
45
46     def resolve_workspace_root(workspace_root: str, output_root: str, output_dir: str) ->
str:
47         """Durable workspace root, by precedence: ``storage.workspace_root`` ?
48         ``storage.output_root`` ? ``output.output_dir``. The first non-empty wins (non-
breaking)."""
49         return (workspace_root or "").strip() or (output_root or "").strip() or (output_dir
or "output")
50
51
52     @dataclass(frozen=True)
53     class VideoWorkspace:
54         """The single folder for one video. Pure path math; build with :func:`for_video`.
55
56         ``base`` = ``<root>/[<prefix>]/<video_id>``. Subfolder accessors return URIs/paths
under it.
57         """
58
59         root: str
60         video_id: str
61         prefix: str = "workspaces"
62
63         @property
64         def base(self) -> str:
65             if self.prefix:
66                 return join_uri(self.root, self.prefix, self.video_id)
67             return join_uri(self.root, self.video_id)
68
69         def path(self, sub: str = "", name: str = "") -> str:
70             """A path under this video's folder: ``base[/sub][/name]``."""
71             return join_uri(self.base, sub, name)
72
73         @property
74         def source(self) -> str:
75             return self.path("source")
76
77         @property
78         def detector(self) -> str:
79             return self.path("detector")
80
81         @property
82         def outputs(self) -> str:
83             return self.path("outputs")
84
85         @property
86         def reels(self) -> str:
87             return self.path("reels")
88
89         @property
90         def logs(self) -> str:
91             return self.path("logs")
92
93         @property
94         def tmp(self) -> str:
95             return self.path("tmp")
96
97

```

```

98     def for_video(root: str, video_id: str, prefix: str = "workspaces") -> Videoworkspace:
99         """Build the :class:`Videoworkspace` for ``video_id`` under ``root`` (a storage
URI)."""
100         if not video_id:

```

11. user (2026-06-22T16:12:29.606Z)

```

diff --git a/backend/config/models.py b/backend/config/models.py
index d77f98e..9c8cf77 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -384,6 +384,26 @@ class BillingConfig(BaseModel):
     margin: float = 0.0                                # resale markup (?0); 0 = bill at cost

+class FlywheelConfig(BaseModel):
+    """Data-flywheel harness (COMPUTE_DECOUPLED_SERVING M11, D12) ? the
consent?capture?eval?goldenize
+    loop: on a CONSENTED adult job, durably **capture** the source + rally output into the per-
video
+    workspace ? **shadow-eval** owned-vs-Gemini ? **propose promotion** to the golden TRAINING
set, so
+    the owned model climbs toward the D11 ship floor.
+
+    **DEFAULT-OFF + INERT:** ``enabled=False`` ? nothing is captured/evaluated (byte-identical
to
+    today). ? **Consent is FAKED-DENY until counsel + the consent schema land:** even
``enabled=True``
+    captures **nothing**, because ``flywheel.consent_attested`` currently returns ``False`` for
every
+    job (the privacy guardrail ? D10: adults-only, explicit opt-in, derived-data + auto-delete-
raw).
+    This is the PLUMBING only; the real consent gate (counsel-reviewed ToS/DPA ? consent
columns ?
+    wiring) + the owned-model + golden-label data needed to *activate* shadow-eval/promotion
are
+    owner/counsel/GPU scope. Pairs with M9-consent."""
+
+    enabled: bool = False
+    workspace_prefix: str = "flywheel"                # per-video workspace prefix for captured artifacts
+    shadow_eval: bool = True                          # run the owned-vs-Gemini shadow comparison on capture
+    min_promote_n: int = 5                            # promote_if_served_safe `min_n` (the served-safe
gate)
+
+
class AppConfig(BaseModel):
    """Root configuration object.

@@ -406,6 +426,7 @@ class AppConfig(BaseModel):
    storage: StorageConfig = Field(default_factory=StorageConfig)
    deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
    billing: BillingConfig = Field(default_factory=BillingConfig)
+    flywheel: FlywheelConfig = Field(default_factory=FlywheelConfig)

    # Allow extra keys during transition so existing config.json files do not break.
    model_config = {

```

12. assistant (2026-06-22T16:12:34.171Z)

Let me now check the storage.py to verify the signature and behavior:

13. user (2026-06-22T16:12:35.158Z)

```

114: def put(local_path: "str | os.PathLike[str]", uri: str, *,

```

14. user (2026-06-22T16:12:37.883Z)

```
1      """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3      URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4      lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5      ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6      today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7      Google Drive (Google Drive for Desktop) as plain files.
8
9      Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10     ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11     passthrough. See ``docs/DECOUPLED_COMPUTE``.
12     """
13     from __future__ import annotations
14
15     import os
16     import tempfile
17     from typing import Any, Dict, Iterator, Optional
18
19     from backend.storage.base import StorageBackend
20     from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
21
22     __all__ = [
23         "StorageBackend", "StorageRef", "StorageStat", "parse_uri", "resolve_input_uri",
24         "get_storage", "fetch", "put", "list_uris",
25         "acquire_local_input", "input_is_local", "resolve_input_ref",
26     ]
27
28
29     def _storage_dict(storage_config: Any) -> Dict[str, Any]:
30         """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
31         if storage_config is None:
32             return {}
33         if hasattr(storage_config, "model_dump"):
34             return storage_config.model_dump() # type: ignore[no-any-return]
35         return dict(storage_config)
36
37
38     def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
39         """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
40         config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
41         callers at the API/CLI boundary catch that and return a clean client error (never a
500).
42
43         Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
44         ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
45         sc = _storage_dict(storage_config)
46         uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
47                                input_root=sc.get("input_root", ""))
48         return parse_uri(uri)
49
50
51     def input_is_local(raw: str, storage_config: Any = None) -> bool:
52         """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
53         plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
```

```

URI
54         that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
55         ``resolve_input_ref``)."
56         return resolve_input_ref(raw, storage_config).backend == "local"
57
58
59     def acquire_local_input(raw: str, storage_config: Any = None, *,
60                             scratch_parent: "str | None" = None) -> str:
61         """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
62
63         Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
64         byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
65         non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
66         path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
67         regenerable from the source) and relies on OS temp reclamation ? the same pull
pattern the
68         worker's ``cmd_infer`` already ships.""
69         sc = _storage_dict(storage_config)
70         ref = resolve_input_ref(raw, sc)
71         if ref.backend == "local":
72             return ref.key # identity ? no temp, no copy, byte-identical to passing the
path directly
73         scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
74         return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
75
76
77     def _backend_class(name: str):
78         """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
79         if name == "local":
80             from backend.storage.local import LocalStorage
81             return LocalStorage
82         if name == "s3":
83             from backend.storage.s3 import S3Storage
84             return S3Storage
85         if name == "gcs":
86             from backend.storage.gcs import GCSStorage
87             return GCSStorage
88         if name == "gdrive":
89             from backend.storage.gdrive import GDriveStorage
90             return GDriveStorage
91         raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive)")
92
93
94     def get_storage(backend: Optional[str] = None, *,
95                     config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
96         """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
97         the implementation; its per-backend settings come from ``config[backend]`` merged
with
98         ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors.""
99         cfg = config or {}
100         name = backend or cfg.get("backend", "local")
101         settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
102         settings.update(overrides)
103         return _backend_class(name)(**settings)
104
105
106     def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,

```

```

107         config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
108         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
109         (no copy); for s3/gcs it downloads to ``dst``."""
110         ref = parse_uri(uri)
111         return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)
112
113
114     def put(local_path: "str | os.PathLike[str]", uri: str, *,
115            config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
str:
116         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
117         ref = parse_uri(uri)
118         get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
119         return uri
120
121
122     def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->
Iterator[str]:
123         """Yield the URI of every object under ``prefix_uri``."""
124         ref = parse_uri(prefix_uri)
125         for r in get_storage(ref.backend, config=config).list(ref):
126             yield r.uri
127

```

15. assistant (2026-06-22T16:12:40.568Z)

Now let me verify the existing config tests to see if adding FlywheelConfig breaks anything:

16. user (2026-06-22T16:12:41.508Z)

```

tests/test_api_cloud_dispatch.py:    cfg = AppConfig() # compute_target defaults to ""
tests/test_api_endpoints.py:    cfg = AppConfig()
tests/test_api_endpoints.py:    cfg = AppConfig()
tests/test_api_endpoints.py:    cfg = AppConfig()
tests/test_api_endpoints.py:    cfg = AppConfig()
tests/test_api_endpoints.py:    cfg = AppConfig()
tests/test_async_jobs.py:    cfg = AppConfig()
tests/test_billing.py:    monkeypatch.setattr(bmain, "global_config", AppConfig(),
raising=False) # billing.enabled=False
tests/test_bind_and_host.py:    assert m._resolve_bind_host(AppConfig(), None) == "127.0.0.1"
tests/test_bind_and_host.py:    cfg = AppConfig()
tests/test_bind_and_host.py:    assert m._resolve_bind_host(AppConfig(), None) == "0.0.0.0"
tests/test_bind_and_host.py:    cfg = AppConfig()
tests/test_config.py:    default_dump = AppConfig().model_dump(mode="json")
tests/test_config.py:    cfg = AppConfig()
tests/test_config.py:    cfg = AppConfig()
tests/test_config_deployment.py:    AppConfig()'s dump, and the new section is present with
empty values."""
tests/test_config_deployment.py:    assert dumped == AppConfig().model_dump(mode="json")
tests/test_config_deployment.py:    merged = {**AppConfig().model_dump(mode="json")}
tests/test_config_wasb.py:    idx = AppConfig().get("indexing")
tests/test_config_wasb.py:    assert AppConfig().get("indexing")["ai_chunk_scale_width"] == 1280

```

17. assistant (2026-06-22T16:12:43.295Z)

Let me check for config field count tests:

18. user (2026-06-22T16:12:44.168Z)

```

tests/test_config.py:- Old config files with extra keys are tolerated.
tests/test_config.py:    # Unspecified keys fall back to defaults

```

```

tests/test_config.py: def test_extra_keys_tolerated():
tests/test_config.py:     """Old or future keys in config.json must not break loading."""
tests/test_config.py: def test_unknown_keys_warn_with_dotted_paths(tmp_path, caplog):
tests/test_config.py:     """Typo'd keys must be LOUD: top-level extras are kept-but-unread and
unknown
tests/test_config.py:     section keys are silently dropped, so without this warning a typo just
makes
tests/test_config.py:     (no deep merge of sub-keys), per loader's {**defaults, **file_data}.
tests/test_config.py:     """Positive test for Item 2: _unknown_key_paths warning for unknown
top-level keys,
tests/test_config.py:     but tolerated (extra='allow'), and unknown section keys dropped
silently.
tests/test_config.py:     missing key fallbacks, and leaf keys discoverable inside sections.
tests/test_config.py:     # leaf keys inside any section resolve via the dynamic section scan
(not a hardcoded
tests/test_config.py:     # list), so a future top-level section's keys would be discoverable too
tests/test_config.py:     # not the raw model, so a future secret field can't leak. Non-secret
fields still serialize.
tests/test_config_deployment.py: preset sub-key while sibling preset/default sub-keys survive ?
i.e. real deep-merge layering).
tests/test_config_deployment.py:     literal_profiles =
set(typing.get_args(DeploymentConfig.model_fields["profile"].annotation))
tests/test_config_deployment.py:     literal_targets =
set(typing.get_args(DeploymentConfig.model_fields["compute_target"].annotation))
tests/test_config_wasb.py: nested sections silently dropped unknown keys and the whole
indexing.wasb block (the live
tests/test_config_wasb.py:     fields = IndexingConfig.model_fields
tests/test_config_wasb.py:     assert "yolo_prefetch_workers" in fields

```

19. assistant (2026-06-22T16:12:46.409Z)

Let me check the M8 cost ledger hook to compare the pattern:

20. user (2026-06-22T16:12:47.038Z)

```

diff --git a/backend/api/job_worker.py b/backend/api/job_worker.py
index 50642ed..aae876c 100644
--- a/backend/api/job_worker.py
+++ b/backend/api/job_worker.py
@@ -89,6 +89,7 @@ def _run_claimed_job(job: Dict[str, Any]) -> None:
     db_instance.set_job_video_id(job_id, result["video_id"])
     db_instance.mark_job_done(job_id, result)
     _maybe_record_job_cost(job, result) # M8 cost ledger (default-OFF)
+    _maybe_run_flywheel(job, result)    # M11 data-flywheel (default-OFF + consent-faked-
deny)
     except _main.JobWaiting as w:
         # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
         logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
@@ -129,6 +130,28 @@ def _maybe_record_job_cost(job: Dict[str, Any], result: Dict[str, Any]) ->
None:
     logger.warning("Cost recording for job %s failed (non-fatal): %s", job.get("id"), e)

+def _maybe_run_flywheel(job: Dict[str, Any], result: Dict[str, Any]) -> None:
+    """M11 data-flywheel (COMPUTE_DECOUPLED_SERVING M11, D12). DEFAULT-OFF + INERT: a no-op
unless
+    + ``config.flywheel.enabled`` AND attested consent ? and consent is a **faked hard-deny**
today
+    + (``flywheel.consent_attested`` returns False for every job until counsel + the consent
schema
+    + land), so this captures **nothing** in production. When activated it captures the consented
job's
+    + artifacts into the per-video workspace (? later labeling ? the golden training set).
+    +
+    + Best-effort ? never fails a completed job because the flywheel hiccuped. ``job`` is the

```



```

claimed
+   row (carries ``owner_id``); ``result`` is the pipeline output."""
+   import backend.main as _main
+
+   flywheel_cfg = getattr(_main.global_config, "flywheel", None)
+   if not flywheel_cfg or not getattr(flywheel_cfg, "enabled", False):
+       return
+   try:
+       from backend import flywheel
+
+       flywheel.run_for_job(job, result, _main.global_config)
+   except Exception as e: # never wedge a completed job on the flywheel
+       logger.warning("Flywheel for job %s failed (non-fatal): %s", job.get("id"), e)
+
+
+   def _drain_due_jobs() -> int:
+       """One worker tick: claim and run every currently-runnable job (queued, or waiting whose
+       `next_poll_at` is due), earliest-due first, until the table has nothing runnable.

```

21. assistant (2026-06-22T16:12:48.978Z)

Now let me check if there are any existing DB schema references to consent that don't exist yet:

22. user (2026-06-22T16:12:50.075Z)

```

backend/flywheel.py:The consent?capture?eval?goldenize loop: on a **consented adult** job,
durably **capture** the source
backend/flywheel.py: 2. :func:`consent_attested` is a **FAKED hard-deny** (returns ``False``
for every job) until the real
backend/flywheel.py:    consent schema lands (counsel-reviewed ToS/DPA ? consent columns ?
wiring). So even
backend/flywheel.py:`eval.promotion` / ``storage`` code so the loop works the day consent +
the owned model + golden
backend/flywheel.py:labels are wired. It ships **no live behavior change**. Pairs with
M9-consent (the consent columns).
backend/flywheel.py:def consent_attested(job: Dict[str, Any], config: Any = None) -> bool:
backend/flywheel.py:    """The flywheel's consent gate ? currently a **FAKED hard-deny** (always
``False``).
backend/flywheel.py:    consent (D10 / the CLAUDE.md guardrail: minors excluded; per-user
training is a separate explicit
backend/flywheel.py:    opt-in; paid tier never trains). The REAL gate ? reading counsel-defined
consent columns + an
backend/flywheel.py:    adults-only attestation + a retention class ? lands with **M9-consent**;
until then this denies
backend/flywheel.py:    """Durably persist a consented job's artifacts into its per-video
`workspace` (the raw material a
backend/flywheel.py:    ``config.flywheel.enabled`` AND :func:`consent_attested` ? so today it
never captures (consent is
backend/flywheel.py:    faked-deny). On a consented job it captures the artifacts into the per-
video workspace; shadow-eval
backend/flywheel.py:    if not consent_attested(job, config):
backend/flywheel.py:        logger.info("[FLYWHEEL] job %s not captured ? no attested consent
(faked-deny).", job.get("id"))
backend/api/job_worker.py:        _maybe_run_flywheel(job, result)        # M11 data-flywheel
(default-OFF + consent-faked-deny)
backend/api/job_worker.py:    ``config.flywheel.enabled`` AND attested consent ? and consent is
a **faked hard-deny** today
backend/api/job_worker.py:    (`flywheel.consent_attested` returns False for every job until
counsel + the consent schema
backend/api/job_worker.py:    land), so this captures **nothing** in production. When activated
it captures the consented job's
tests/test_flywheel.py:**faked consent hard-deny** (`consent_attested` returns False for every
job ? nothing is captured
tests/test_flywheel.py:# -----
default-OFF + consent guardrail

```

```

tests/test_flywheel.py: def test_consents_is_faked_hard_deny():
tests/test_flywheel.py:     """The privacy guardrail: consent_attested denies EVERY job until
counsel + the schema land."""
tests/test_flywheel.py:     assert flywheel.consent_attested({"id": 1}, _cfg(enabled=True)) is
False
tests/test_flywheel.py:     assert flywheel.consent_attested({"id": 2, "owner_id": "u@x"}, None)
is False
tests/test_flywheel.py: def test_run_for_job_noop_when_enabled_but_no_consent(tmp_path):
tests/test_flywheel.py:     """enabled=True but consent denied (the faked default) ? still a no-
op, NOTHING written."""
tests/test_flywheel.py: #
----- capture (consented)
tests/test_flywheel.py: def test_run_for_job_captures_when_enabled_and_consented(tmp_path,
monkeypatch):
tests/test_flywheel.py:     """With consent monkeypatched True, a per-video manifest lands under
<output>/flywheel/<vid>/."""
tests/test_flywheel.py:     monkeypatch.setattr(flywheel, "consent_attested", lambda job,
config=None: True)
tests/test_flywheel.py:     monkeypatch.setattr(flywheel, "consent_attested", lambda job,
config=None: True)

```

23. assistant (2026-06-22T16:12:52.000Z)

Let me check the database code to confirm no DB references are made:

24. user (2026-06-22T16:12:53.386Z)

(Bash completed with no output)

25. assistant (2026-06-22T16:12:55.579Z)

Now let me verify the promotion.py argument order one more time for the critical A/B issue:

26. user (2026-06-22T16:12:56.412Z)

```

def promote_if_served_safe(
    served_a: Sequence[float],
    served_b: Sequence[float],
    *,
    proposal_a: Optional[Sequence[float]] = None,
    proposal_b: Optional[Sequence[float]] = None,
    structural: bool = False,
    min_n: int = 5,
    p_threshold: float = 0.05,
    served_regress_eps: float = 0.0,
    n_boot: int = 2000,
    seed: int = 0,
) -> ServedGateResult:
    """Promote a cue ONLY if it does not regress at the SERVED windowing (the foundational
rule).

```

The honest bar (`should_promote_for_user`: min-N floor + ?F1 CI excludes 0 + sign test) is applied to the `served` scores ? the operating point production runs ? so a proposal-only win can never carry a promotion. The optional `proposal_*` scores are scored too and reported alongside, purely to surface the eval/serve contrast (they NEVER promote on their own).

27. user (2026-06-22T16:12:59.082Z)

```

130
131     Args:
132         served_a/served_b: per-video held-out scores for variant A (control) and B (the
cue),
133         measured at the SERVED windowing. These drive the decision.

```

```

134         proposal_a/proposal_b: the same scores at the PROPOSAL windowing (optional).
Reported for
135         contrast; the served regression guard still has the final say.
136         structural: forwarded to ``should_promote_for_user`` ? a structural guard (e.g.
Gate-A vs
137         held-shuttle FPs) is never *disabled* on "no lift", but it is ALSO not
auto-*promoted* to
138         the served default if it regresses served. So for ``structural=True`` we report
139         ``keep_on`` honestly but ``promote`` reflects the served-safety check.
140         served_regress_eps: served regression tolerance (0.0 = "no served-mean drop
allowed").
141
142         Returns a :class:`ServedGateResult` with the final verdict + both windowings'
evidence.
143         """
144         # 1) The honest gate, applied to the SERVED scores (never the proposal ones).

```

28. assistant (2026-06-22T16:13:01.072Z)

Now let me verify the flywheel.propose_promotion call and the argument order:

29. user (2026-06-22T16:13:01.929Z)

```

107     def propose_promotion(
108         served_owned: Sequence[float],
109         served_gemini: Sequence[float],
110         *,
111         min_n: int = 5,
112     ) -> Any:
113         """Whether the owned model is **served-safe** to promote toward default ? wraps
114         :func:`backend.eval.promotion.promote_if_served_safe` (the foundational *no-
regression-at-served-
115         windowing* gate). ``served_`` are per-video served F1s on the same corpus; Gemini
is the control
116         (A), the owned model the proposal (B)."""
117         from backend.eval.promotion import promote_if_served_safe
118
119         return promote_if_served_safe(served_gemini, served_owned, min_n=min_n)
120
121

```

30. assistant (2026-06-22T16:13:03.902Z)

Perfect! Now let me check for any exception handling issues in capture_job:

31. user (2026-06-22T16:13:04.785Z)

```

54     def capture_job(
55         workspace: "_workspace.VideoWorkspace",
56         *,
57         rally_output: Dict[str, Any],
58         source_path: Optional[str] = None,
59         storage_config: Any = None,
60     ) -> Dict[str, str]:
61         """Durably persist a consented job's artifacts into its per-video ``workspace`` (the
raw material a
62         human later labels ? the golden training set). Always writes a small
``capture.json`` manifest
63         (``video_id`` + rally output + provenance); when a local ``source_path`` is given,
also stores the
64         source video. Returns the written URIs. Reuses :func:`backend.storage.put` + the
``VideoWorkspace``
65         layout, so it honours whatever ``storage.backend`` the deploy uses (local / gcs /
s3)."""

```

```

66         written: Dict[str, str] = {}
67         cfg = _sc(storage_config)
68         fd, manifest_local = tempfile.mkstemp(suffix=".json")
69         try:
70             with os.fdopen(fd, "w", encoding="utf-8") as fh:
71                 json.dump({"video_id": workspace.video_id, "rally_output": rally_output},
72 fh, indent=2)
73                 manifest_uri = _workspace.join_uri(workspace.outputs, "capture.json")
74                 written["manifest_uri"] = _storage.put(manifest_local, manifest_uri, config=cfg)
75                 if source_path and os.path.exists(source_path):
76                     source_uri = _workspace.join_uri(workspace.source,
77 os.path.basename(source_path))
78                     written["source_uri"] = _storage.put(source_path, source_uri, config=cfg)
79         finally:
80             try:
81                 os.unlink(manifest_local)
82             except OSError:
83                 pass
84         return written

```

32. assistant (2026-06-22T16:13:06.797Z)

Now let me check if there are any tests that break with the new FlywheelConfig field:

33. user (2026-06-22T16:13:10.749Z)

```

===== test session starts =====
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\m11-flywheel
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 13 items

tests/test_flywheel.py::test_flywheel_config_defaults_off PASSED [ 7%]
tests/test_flywheel.py::test_consent_is_faked_hard_deny PASSED [ 15%]
tests/test_flywheel.py::test_run_for_job_noop_when_disabled PASSED [ 23%]
tests/test_flywheel.py::test_run_for_job_noop_when_enabled_but_no_consent PASSED [ 30%]
tests/test_flywheel.py::test_run_for_job_captures_when_enabled_and_consented PASSED [ 38%]
tests/test_flywheel.py::test_run_for_job_no_video_id_is_noop PASSED [ 46%]
tests/test_flywheel.py::test_capture_job_persists_manifest_and_source PASSED [ 53%]
tests/test_flywheel.py::test_shadow_eval_labeled_delegates_to_compare PASSED [ 61%]
tests/test_flywheel.py::test_shadow_eval_unlabeled_delegates_to_compare_unlabeled PASSED [ 69%]
tests/test_flywheel.py::test_shadow_eval_unlabeled_requires_single_video PASSED [ 76%]
tests/test_flywheel.py::test_propose_promotion_delegates_with_gemini_as_control PASSED [ 84%]
tests/test_flywheel.py::test_job_worker_hook_noop_when_disabled PASSED [ 92%]
tests/test_flywheel.py::test_job_worker_hook_is_best_effort PASSED [100%]

===== 13 passed in 2.02s =====

```

34. user (2026-06-22T16:13:14.705Z)

```

===== test session starts =====
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\m11-flywheel
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... ERROR: Wrong expression passed to '-k': extra_keys\|AppConfig: at column 12:
unexpected character "|"

```

collected 21 items

===== no tests ran in 0.27s =====

35. user (2026-06-22T16:13:18.382Z)

===== test session starts =====

platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\m11-flywheel
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 1 item

tests/test_config.py::test_extra_keys_tolerated PASSED [100%]

===== 1 passed in 0.29s =====

36. user (2026-06-22T16:13:25.233Z)

tests/test_async_jobs.py::test_migration_upgrades_v1_db_in_place PASSED [14%]
tests/test_async_jobs.py::test_parity_job_model_with_direct_processing PASSED [21%]
tests/test_async_jobs.py::test_submit_returns_202_with_job_id PASSED [28%]
tests/test_async_jobs.py::test_submit_rejects_null_byte_path PASSED [35%]
tests/test_async_jobs.py::test_submit_missing_file_is_404 PASSED [42%]
tests/test_async_jobs.py::test_submit_unknown_segmenter_400_and_no_job_row PASSED [50%]
tests/test_async_jobs.py::test_job_done_with_segments_and_video_id PASSED [57%]
tests/test_async_jobs.py::test_job_done_but_pipeline_verdict_failed_preserves_prior PASSED [64%]
tests/test_async_jobs.py::test_worker_crash_marks_failed_and_restores_prior_segments PASSED [71%]
tests/test_async_jobs.py::test_progress_stages_are_recorded PASSED [78%]
tests/test_async_jobs.py::test_get_unknown_job_is_404 PASSED [85%]
tests/test_async_jobs.py::test_fail_stale_jobs_terminalizes_queued_and_running_only PASSED [92%]
tests/test_async_jobs.py::test_real_executor_completes_job PASSED [100%]

===== 14 passed in 3.46s =====

37. assistant (2026-06-22T16:13:27.221Z)

Now let me check if the tempfile fd handling in capture_job is correct:

38. user (2026-06-22T16:13:31.293Z)

OK: fd closed properly by fdopen
OK: fd cleanup works in exception path

39. assistant (2026-06-22T16:13:33.615Z)

Perfect! Now let me run the full test suite to ensure nothing breaks:

40. user (2026-06-22T16:14:55.496Z)

=====

Full test suite: C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest -q
obsreport (optional reporting dep): available

=====

.....SS..... [4%]
..... [9%]
..... [14%]
..... [19%]

```

..... [ 24%]
..... [ 29%]
..... [ 34%]
..... [ 39%]
..... [ 44%]
..... [ 49%]
..... [ 54%]
..... [ 59%]
..... [ 64%]
..... [ 69%]
..... [ 74%]
.....SSSSS..... [ 79%]
..... [ 84%]
..... [ 89%]
..... [ 94%]
..... [ 99%]
..... [100%]
1447 passed, 7 skipped in 78.34s (0:01:18)

```

```

=====
RESULT: PASS - all tests green
=====

```

41. assistant (2026-06-22T16:14:57.611Z)

Excellent! Let me check ruff and mypy:

42. user (2026-06-22T16:14:58.868Z)

All checks passed!

43. user (2026-06-22T16:15:23.794Z)

```

backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]

```

44. assistant (2026-06-22T16:15:25.923Z)

Good! No real errors. Let me verify the workspace properties are accessed without parens:

45. user (2026-06-22T16:15:26.813Z)

```

72:         manifest_uri = _workspace.join_uri(workspace.outputs, "capture.json")
75:         source_uri = _workspace.join_uri(workspace.source, os.path.basename(source_path))

```

46. assistant (2026-06-22T16:15:29.214Z)

Perfect! Now let me double-check the _sc function and storage.put signature match:

47. user (2026-06-22T16:15:29.899Z)

```

33     def _sc(storage_config: Any) -> Optional[Dict[str, Any]]:
34         """Normalize an ``AppConfig.storage`` (pydantic model / dict / None) to the dict
35         ``backend.storage`` expects."""
36         if storage_config is None:
37             return None
38         if hasattr(storage_config, "model_dump"):
39             return storage_config.model_dump()
40         return storage_config if isinstance(storage_config, dict) else None

```

41
42

48. assistant (2026-06-22T16:15:32.275Z)

Perfect. The `_sc` function correctly handles Pydantic models, dicts, and None, and it's used correctly in `capture_job` line 67. Now let me verify the `experiment.compare` and `compare_unlabeled` signatures one more time:

49. user (2026-06-22T16:15:33.172Z)

```
def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
            iou: float = 0.5, honest: bool = True, honest_stats: bool = True) -> Dict[str, Any]:
    """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the same
    golden corpus and reports the B ? A deltas, per video and in aggregate.
```

The deltas use the existing ``metrics.score`` (per video) + ``calibration.pct_improvement``;

50. user (2026-06-22T16:15:35.820Z)

```
def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
                      agree_iou: float = 0.5) -> Dict[str, Any]:
    """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
```

51. assistant (2026-06-22T16:15:41.887Z)

Perfect. Now let me check the `shadow_eval` function signatures and verify they match:

52. user (2026-06-22T16:15:42.546Z)

```
85     def shadow_eval(
86         candidates: Sequence[Any],
87         owned_variant: Any,
88         gemini_variant: Any,
89         *,
90         labeled: bool = False,
91         iou: float = 0.5,
92     ) -> Dict[str, Any]:
93         """Owned-vs-Gemini shadow comparison (the D11 climb signal). Delegates to the
experiment harness:
94         a labeled golden corpus ? :func:`backend.eval.experiment.compare` (B?A deltas +
honest CIs); an
95         unlabeled single video ? :func:`backend.eval.experiment.compare_unlabeled`
(agreement). The owned
96         model + a golden corpus are owner/GPU-side, so this only *runs* once they exist; the
wrapper makes
97         the call site real today (and is exercised via tests with stub variants)."""
98         from backend.eval import experiment
99
100        if labeled:
101            return experiment.compare(candidates, owned_variant, gemini_variant, iou=iou)
102        if len(candidates) != 1:
103            raise ValueError("unlabeled shadow_eval compares exactly one video at a time")
104        return experiment.compare_unlabeled(candidates[0], owned_variant, gemini_variant,
agree_iou=iou)
```

53. assistant (2026-06-22T16:15:44.856Z)

Perfect! Let me verify the test captures the delegation correctly:

54. user (2026-06-22T16:15:45.626Z)

```
88     def test_shadow_eval_labeled_delegates_to_compare(monkeypatch):
89         calls = {}
```

```

90     from backend.eval import experiment
91
92     def fake_compare(c, a, b, iou=0.5):
93         calls["compare"] = (c, a, b, iou)
94         return {"ok": 1}
95
96     monkeypatch.setattr(experiment, "compare", fake_compare)
97     res = flywheel.shadow_eval(["c1", "c2"], "owned", "gemini", labeled=True, iou=0.4)
98     assert res == {"ok": 1}
99     assert calls["compare"] == ([ "c1", "c2"], "owned", "gemini", 0.4)
100
101
102     def test_shadow_eval_unlabeled_delegates_to_compare_unlabeled(monkeypatch):
103         from backend.eval import experiment
104         seen = {}
105
106         def fake_unlabeled(v, a, b, agree_iou=0.5):
107             seen["u"] = (v, a, b, agree_iou)
108             return {"agree": 1}
109
110         monkeypatch.setattr(experiment, "compare_unlabeled", fake_unlabeled)
111         assert flywheel.shadow_eval(["only"], "owned", "gemini") == {"agree": 1}
112         assert seen["u"] == ("only", "owned", "gemini", 0.5)
113
114

```

55. assistant (2026-06-22T16:15:47.737Z)

Good! Now let me check the test for propose_promotion one more time to verify A/B order:

56. user (2026-06-22T16:15:48.603Z)

```

120     def test_propose_promotion_delegates_with_gemini_as_control(monkeypatch):
121         """The owned model is the PROPOSAL (B); Gemini is the CONTROL (A) ? promote only if
served-safe."""
122         import backend.eval.promotion as promotion
123         seen = {}
124
125         def fake_gate(a, b, *, min_n=5):
126             seen["p"] = (list(a), list(b), min_n)
127             return "VERDICT"
128
129         monkeypatch.setattr(promotion, "promote_if_served_safe", fake_gate)
130         out = flywheel.propose_promotion([0.8, 0.9], [0.7, 0.75], min_n=6) # owned, gemini
131         assert out == "VERDICT"
132         assert seen["p"] == ([0.7, 0.75], [0.8, 0.9], 6) # A=gemini(control),
B=owned(proposal)
133

```

57. assistant (2026-06-22T16:15:52.491Z)

Perfect! The test clearly shows:

- `flywheel.propose_promotion([0.8, 0.9], [0.7, 0.75])` receives owned first, gemini second`
- Line 119 correctly swaps them: `promote_if_served_safe(served_gemini, served_owned, ...)``
- The test verifies: `seen["p"] == ([0.7, 0.75], [0.8, 0.9], 6)` meaning Gemini (control/A) = [0.7, 0.75] and owned (proposal/B) = [0.8, 0.9]`

Now let me verify the capture_job missing source_path handling and the storage.put behavior:

58. user (2026-06-22T16:15:53.423Z)

```

54     def capture_job(
55         workspace: "_workspace.VideoWorkspace",
56         *,

```



```

57         rally_output: Dict[str, Any],
58         source_path: Optional[str] = None,
59         storage_config: Any = None,
60     ) -> Dict[str, str]:
61         """Durably persist a consented job's artifacts into its per-video ``workspace`` (the
raw material a
62         human later labels ? the golden training set). Always writes a small
``capture.json`` manifest
63         (``video_id`` + rally output + provenance); when a local ``source_path`` is given,
also stores the
64         source video. Returns the written URIs. Reuses :func:`backend.storage.put` + the
``VideoWorkspace``
65         layout, so it honours whatever ``storage.backend`` the deploy uses (local / gcs /
s3)."""
66         written: Dict[str, str] = {}
67         cfg = _sc(storage_config)
68         fd, manifest_local = tempfile.mkstemp(suffix=".json")
69         try:
70             with os.fdopen(fd, "w", encoding="utf-8") as fh:
71                 json.dump({"video_id": workspace.video_id, "rally_output": rally_output},
fh, indent=2)
72                 manifest_uri = _workspace.join_uri(workspace.outputs, "capture.json")
73                 written["manifest_uri"] = _storage.put(manifest_local, manifest_uri, config=cfg)
74                 if source_path and os.path.exists(source_path):
75                     source_uri = _workspace.join_uri(workspace.source,
os.path.basename(source_path))
76                     written["source_uri"] = _storage.put(source_path, source_uri, config=cfg)
77             finally:
78                 try:
79                     os.unlink(manifest_local)
80                 except OSError:
81                     pass
82             return written
83

```

59. assistant (2026-06-22T16:15:59.367Z)

Perfect. The error handling is solid:

- Line 74: Gracefully checks if source_path exists before attempting to store it
- Lines 77-81: The finally block ensures cleanup of the tempfile even if storage.put() raises
- os.unlink() wrapped in try/except to handle race conditions (file deleted by another process)

Now let me verify the entire control flow from the job_worker hook to make sure it's truly best-effort:

60. user (2026-06-22T16:16:00.147Z)

```

<tool_use_error>InputValidationError: Read failed due to the following issue:
The parameter `offset` type is expected as `number` but provided as `array`</tool_use_error>

```

61. user (2026-06-22T16:16:02.524Z)

```

130         logger.warning("Cost recording for job %s failed (non-fatal): %s",
job.get("id"), e)
131
132
133     def _maybe_run_flywheel(job: Dict[str, Any], result: Dict[str, Any]) -> None:
134         """M11 data-flywheel (COMPUTE_DECOUPLED_SERVING M11, D12). DEFAULT-OFF + INERT: a
no-op unless
135         ``config.flywheel.enabled`` AND attested consent ? and consent is a **faked hard-
deny** today
136         (``flywheel.consent_attested`` returns False for every job until counsel + the
consent schema
137         land), so this captures **nothing** in production. When activated it captures the

```

```

consented job's
138     artifacts into the per-video workspace (? later labeling ? the golden training set).
139
140     Best-effort ? never fails a completed job because the flywheel hiccuped. ``job`` is
the claimed
141     row (carries ``owner_id``); ``result`` is the pipeline output.""
142     import backend.main as _main
143
144     flywheel_cfg = getattr(_main.global_config, "flywheel", None)
145     if not flywheel_cfg or not getattr(flywheel_cfg, "enabled", False):
146         return
147     try:
148         from backend import flywheel
149
150         flywheel.run_for_job(job, result, _main.global_config)
151     except Exception as e: # never wedge a completed job on the flywheel
152         logger.warning("Flywheel for job %s failed (non-fatal): %s", job.get("id"), e)
153
154

```

62. assistant (2026-06-22T16:16:04.841Z)

Perfect. Best-effort wrapper with exception catch. Now let me verify the test that checks the hook is wired correctly:

63. user (2026-06-22T16:16:05.557Z)

```

136
137     def test_job_worker_hook_noop_when_disabled(monkeypatch):
138         """_maybe_run_flywheel must not call into the harness when flywheel is disabled."""
139         import backend.main as bmain
140         from backend.api import job_worker
141
142         monkeypatch.setattr(bmain, "global_config", _cfg(enabled=False), raising=False)
143         called = []
144         monkeypatch.setattr(flywheel, "run_for_job", lambda *a, **k: called.append(1))
145         job_worker._maybe_run_flywheel({"id": 1}, {"video_id": "v"})
146         assert called == []
147
148
149     def test_job_worker_hook_is_best_effort(monkeypatch):
150         """An exception inside the flywheel never propagates (a completed job must not be
wedged)."""
151         import backend.main as bmain
152         from backend.api import job_worker
153
154         monkeypatch.setattr(bmain, "global_config", _cfg(enabled=True), raising=False)
155
156         def _boom(*a, **k):
157             raise RuntimeError("kaboom")
158
159         monkeypatch.setattr(flywheel, "run_for_job", _boom)
160         job_worker._maybe_run_flywheel({"id": 1}, {"video_id": "v"}) # must NOT raise

```

64. assistant (2026-06-22T16:16:07.753Z)

Excellent! The tests verify both that the hook doesn't run when disabled, and that exceptions are swallowed (best-effort). Now let me verify the migration number comment in the changelog:

65. user (2026-06-22T16:16:09.064Z)

| M8 | Per-job **cost ledger** (v6 migration: `owner\_id, gpu\_active\_seconds, wall\_seconds, bytes\_out, cost\_usd, cost\_breakdown\_json`) + **pricebook** + aggregation + `/api/.../cost` | M7
| ? owner (billing) | ? | [#288](https://github.com/Khelsutra/badminton-highlight-

indexer/pull/288) (ledger+pricebook+/cost ? default-OFF; real prices + usage-emission ? owner/follow-up) |

- 2026-06-21 ? **M9-auth code landed** ([#290](https://github.com/Khelsutra/badminton-highlight-indexer/pull/290)): `backend/api/auth.py` ? Cloudflare Access **Cf-Access JWT verify** (RS256: sig+exp+iss+aud against the team JWKS; lazy `PyJWT[crypto]`; injectable JWKS ? fully-offline tests) + `resolve\_tenant` ? tags `create\_job(owner\_id=?)`;

`SecurityConfig.{edge\_auth\_enabled=False,cf\_team\_domain,cf\_access\_aud}` + CORS via `RALLY\_CORS\_ALLOW\_ORIGINS`. **Default-OFF.** Security review (`wf\_9b4c71d0`) found NO bypass; locked alg=none + HS256-confusion rejection. **? M9 ? ? the consent cols + ToS/DPA are owner/counsel; the consent migration is now v7 (M8 took v6).**

- 2026-06-21 ? **M8 code landed** ([#288](https://github.com/Khelsutra/badminton-highlight-indexer/pull/288)): per-job **cost ledger** ? `backend/billing.py` (pure `usage × pricebook` math, USD internal / INR display) + a **v6 migration** (NULLABLE cost columns on `jobs` + a versioned `pricebook` table seeded `placeholder-v0` at \$0; additive=NULL, idempotent) + `record\_job\_cost`/`get\_job\_cost`/`get\_video\_cost` + `GET /api/jobs/{id}/cost` + `/api/videos/{id}/cost` + `BillingConfig` (**default-OFF**) + a gated `\_maybe\_record\_job\_cost` hook. Adversarial review (`wf\_498b24af`) folded a **major silent-GPU-under-bill** (an unpriced `gpu\_type` is now surfaced, not invisible \$0) + the **honest no-op** caveat (usage emission is a follow-up ? cost 0 until wired) + breakdown reconciliation. Suite green, ruff+mypy clean. **? M8 ? ? owner inserts real GCP prices (a new pricebook version) + the worker usage-emission is a follow-up; placeholder ships \$0 = no user-facing change. v6 is the shared bump (M9-consent cols extend it).**

- 2026-06-22 ? **M11 skeleton landed** (data-flywheel harness, D12): `backend/flywheel.py` (`capture\_job` ? per-video `VideoWorkspace` via `backend.storage.put`; `shadow\_eval` ? `experiment.compare`/`compare\_unlabeled`; `propose\_promotion` ? `promotion.promote\_if\_served\_safe` with Gemini=control/owned=proposal; `run\_for\_job` orchestrator) + `FlywheelConfig` (**default-OFF**) + a gated `\_maybe\_run\_flywheel` job-completion hook mirroring M8's cost hook. **TWO independent guards keep it INERT in production:** `flywheel.enabled=False` (default) AND `consent\_attested` is a **FAKED hard-deny** (returns False for every job) ? so even enabled it captures **nothing** until the real consent schema lands. This is the PLUMBING only; it reuses the real workspace/eval/promotion/storage code so it works the day consent + the owned model + golden labels are wired. **NO DB migration** (the skeleton persists no consent). +13 tests (default-OFF no-op, enabled-but-consent-denied no-op, consented-capture writes the manifest, delegation of shadow-eval/promote, the hook is best-effort). Suite green, ruff+mypy clean. **? M11 ? ? LIVE activation is owner/counsel (the consent ToS/DPA + schema ? the consent columns) + owner/GPU (the owned model + golden corpus for shadow-eval/promotion). ? the future M9-consent migration is now `v8` (v7 was taken by the `locale` column).**

66. assistant (2026-06-22T16:16:13.247Z)

Perfect! The changelog correctly states:

- "the future M9-consent migration is now `v8` (v7 was taken by the `locale` column)"
- "NO DB migration (the skeleton persists no consent)"

This is exactly right ? M11 doesn't add a migration because it doesn't read/write any DB columns. The consent columns will be added later in M9-consent. Let me do one final comprehensive verification:

67. user (2026-06-22T16:16:23.973Z)

- ? Test 1: DEFAULT-OFF verified
- ? Test 2: Workspace properties verified
- ? Test 3: Promotion A/B order verified via tests (Gemini=A control, owned=B proposal)
- ? Test 4: Storage config conversion verified
- ? Test 5: Capture_job error handling verified

? All comprehensive checks passed!

68. assistant (2026-06-22T16:16:26.216Z)

Perfect! Now I have all the information I need to produce my findings. Let me compile my security review.

69. user (2026-06-22T16:17:01.171Z)

Output does not match required schema: /inert_guarantee: must be equal to one of the allowed values

70. user (2026-06-22T16:17:20.087Z)

Structured output provided successfully

71. assistant (2026-06-22T16:17:31.372Z)

Summary

****VERDICT: SHIP**** ?

The M11 data-flywheel skeleton is ****correctness-sound and privacy-safe**** for production deployment. Here's what I verified:

Two Independent Safety Guards

1. ****Config default-OFF:**** `AppConfig()` has `flywheel.enabled=False` ? the job-completion hook returns immediately (byte-identical to today).
2. ****Faked consent hard-deny:**** `consent\_attested()` returns `False` for every job, so even if `enabled=True`, nothing is captured until counsel delivers the real consent schema + DB columns.

Correctness Verification

```
| Aspect | Finding |
|-----|-----|
| **Delegation: `propose_promotion` ? `promote_if_served_safe`** | ? Correct A/B order: Gemini=control (A), owned=proposal (B). Arguments swapped at line 119 and verified by test. |
| **Delegation: `shadow_eval` ? `experiment.compare/compare_unlabeled`** | ? Argument order preserved as-is, test-verified. |
| **Workspace properties** | ? `workspace.outputs` and `workspace.source` accessed as `@property` (no parens), correct. |
| **Storage config conversion** | ? `_sc()` correctly converts Pydantic models to dicts that `backend.storage.put()` expects. |
| **`capture_job` error handling** | ? Tempfile cleanup in finally block even if `storage.put()` raises; graceful missing source-path check at line 74. |
| **Job worker hook** | ? Best-effort wrapper: exception caught (line 151), logged, never re-raised. Runs AFTER job marked done in DB. |
| **DB safety** | ? **NO DB migration** ? skeleton references zero DB columns; consent columns land with M9-consent (v8). Changelog correctly documented. |
| **Config test compatibility** | ? Full suite: 1447 passed. Field addition doesn't break any existing tests. |
```

One Nit

The only minor suggestion: add a `test\_consent\_attested\_permanently\_false()` that locks the real (unpatched) `consent\_attested()` to always return `False`, with a comment that this is a counsel-enforced invariant. This prevents accidental relaxation of the hard-deny in future edits.

****Production-ready: YES.**** The skeleton captures nothing today, delegates correctly to real eval/promotion/storage code, and will work immediately when consent schema + owned model + golden labels land.